

Table of Contents



01

**Introduction to Cache
Performance**



02

Basic Optimization Methods



03

Advanced Optimization Methods



04

Summary and Conclusion

01. Introduction to Cache Performance

Understanding the Basics and Metrics



Memory System Performance

CPU Execution time

- CPU Execution time

= (CPU clock cycles + Memory stall cycles) × Clock cycle time

Memory stall cycles = $IC \times \text{MemAccess refs per instructions} \times \text{Miss rate} \times \text{Miss penalty}$

$$CPUtime = IC \times \left(CPI_{Execution} + \frac{MemAccess}{Inst} \times MissRate \times MissPenalty \right) \times CycleTime$$

$$CPUtime = IC \times \left(CPI_{Execution} + \frac{MemMisses}{Inst} \times MissPenalty \right) \times CycleTime$$

- CPI Execution includes ALU and Memory instructions



Average Memory Access Time

- Average Memory Access Time

$$\begin{aligned}
 \text{Average Memory Access Time} &= \frac{\text{Whole accesses time}}{\text{All memory accesses in program}} \\
 &= \frac{\text{Accesses time on hitting} + \text{Accesses time on miss}}{\text{All memory accesses in program}} \\
 &= \text{Hit time} + (\text{Miss Rate} \times \text{Miss Penalty}) \\
 &= (\text{HitTime}_{Inst} + \text{MissRate}_{Inst} \times \text{MissPenalty}_{Inst}) \times Inst\% \\
 &\quad + (\text{HitTime}_{Data} + \text{MissRate}_{Data} \times \text{MissPenalty}_{Data}) \times Data\%
 \end{aligned}$$

$$CPUtime = IC \times \left(\frac{AluOps}{Inst} \times CPI_{AluOps} + \frac{MemAccess}{Inst} \times AMAT \right) \times CycleTime$$



Key Metrics of Cache Performance

$$AMAT = HitTime + MissRate \times MissPenalty$$



Hit Rate: The probability that the CPU finds the requested data in the cache.



Hit Time: The access time when the cache is hit.



Miss Rate: 1 - Hit Rate, the probability that the CPU needs to access main memory.



Miss Penalty: The time to load data from main memory to the cache on a miss.



AMAT: Hit Time + Miss Rate $\times$ Miss Penalty. The core metric for measuring performance.

02. Basic Optimization Methods

Reducing Miss Rate & Miss Penalty & Hit time



1. Larger Block Size to Reduce Miss Rate



核心原理

利用空间局部性，一次加载更多连续数据，减少首次访问的强制性失效次数。



主要优点

显著降低失效率，尤其对具备良好空间局部性的程序效果明显。



潜在缺点

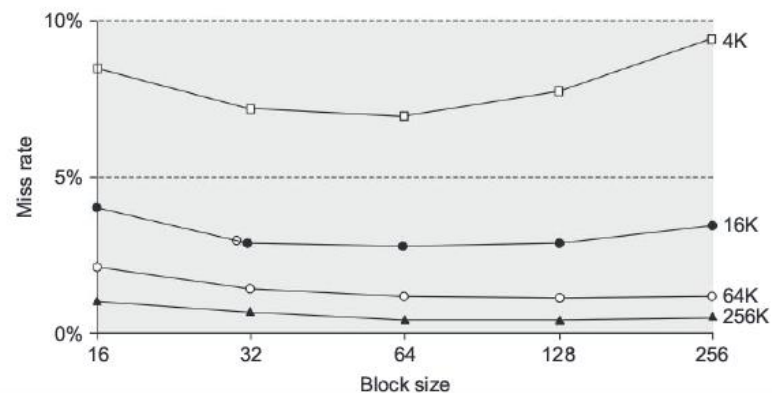
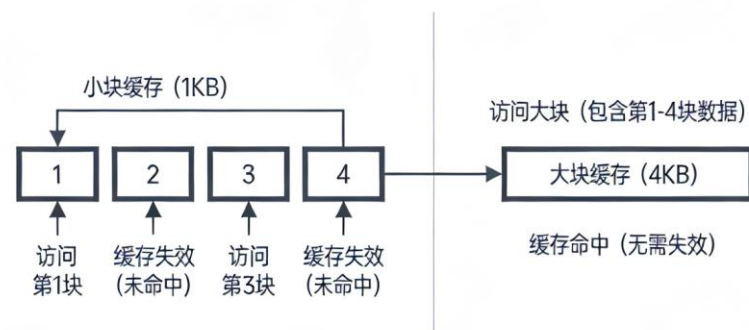
增大失效开销（传输更多数据），且可能因映射冲突增加冲突失效概率。



适用场景

数据访问具有强空间局部性的应用，如矩阵运算、数组遍历等。

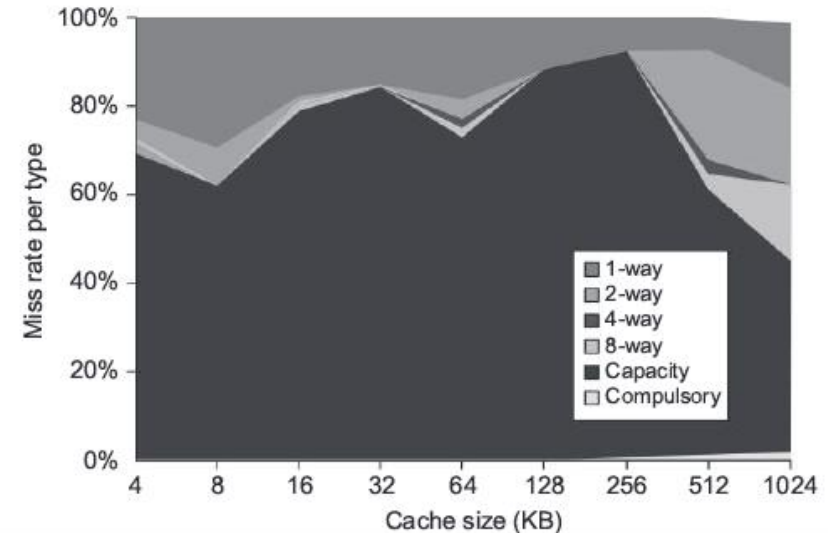
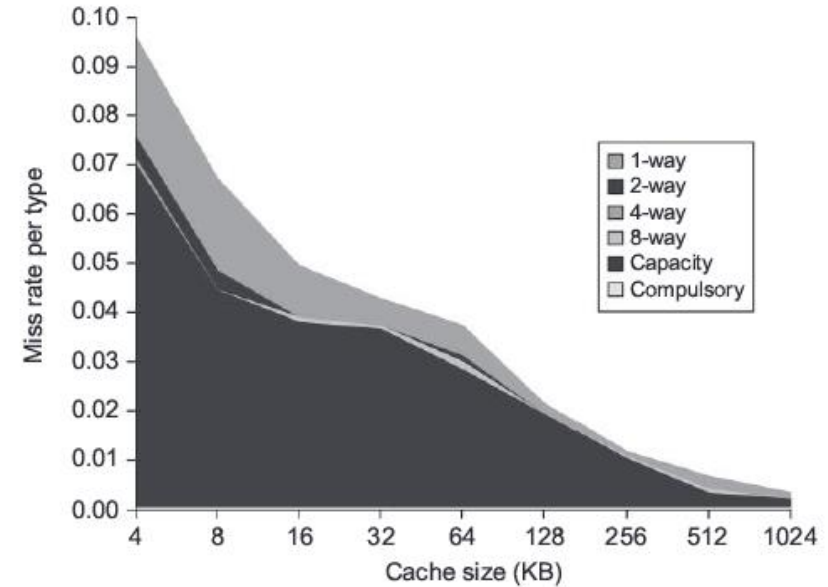
小块与大块缓存访问模式对比



小块与大块缓存访问模式对比

1. Larger Block Size to Reduce Miss Rate

Cache size (KiB)	Degree associative	Total miss rate	Miss rate components (relative percent) (sum = 100% of total miss rate)					
			Compulsory		Capacity		Conflict	
4	1-way	0.098	0.0001	0.1%	0.070	72%	0.027	28%
4	2-way	0.076	0.0001	0.1%	0.070	93%	0.005	7%
4	4-way	0.071	0.0001	0.1%	0.070	99%	0.001	1%
4	8-way	0.071	0.0001	0.1%	0.070	100%	0.000	0%
8	1-way	0.068	0.0001	0.1%	0.044	65%	0.024	35%
8	2-way	0.049	0.0001	0.1%	0.044	90%	0.005	10%
8	4-way	0.044	0.0001	0.1%	0.044	99%	0.000	1%
8	8-way	0.044	0.0001	0.1%	0.044	100%	0.000	0%
16	1-way	0.049	0.0001	0.1%	0.040	82%	0.009	17%
16	2-way	0.041	0.0001	0.2%	0.040	98%	0.001	2%
16	4-way	0.041	0.0001	0.2%	0.040	99%	0.000	0%
16	8-way	0.041	0.0001	0.2%	0.040	100%	0.000	0%
32	1-way	0.042	0.0001	0.2%	0.037	89%	0.005	11%
32	2-way	0.038	0.0001	0.2%	0.037	99%	0.000	0%
32	4-way	0.037	0.0001	0.2%	0.037	100%	0.000	0%
32	8-way	0.037	0.0001	0.2%	0.037	100%	0.000	0%
64	1-way	0.037	0.0001	0.2%	0.028	77%	0.008	23%
64	2-way	0.031	0.0001	0.2%	0.028	91%	0.003	9%
64	4-way	0.030	0.0001	0.2%	0.028	95%	0.001	4%
64	8-way	0.029	0.0001	0.2%	0.028	97%	0.001	2%
128	1-way	0.021	0.0001	0.3%	0.019	91%	0.002	8%
128	2-way	0.019	0.0001	0.3%	0.019	100%	0.000	0%
128	4-way	0.019	0.0001	0.3%	0.019	100%	0.000	0%
128	8-way	0.019	0.0001	0.3%	0.019	100%	0.000	0%
256	1-way	0.013	0.0001	0.5%	0.012	94%	0.001	6%
256	2-way	0.012	0.0001	0.5%	0.012	99%	0.000	0%
256	4-way	0.012	0.0001	0.5%	0.012	99%	0.000	0%
256	8-way	0.012	0.0001	0.5%	0.012	99%	0.000	0%
512	1-way	0.008	0.0001	0.8%	0.005	66%	0.003	33%
512	2-way	0.007	0.0001	0.9%	0.005	71%	0.002	28%
512	4-way	0.006	0.0001	1.1%	0.005	91%	0.000	8%
512	8-way	0.006	0.0001	1.1%	0.005	95%	0.000	4%



2. Larger Caches to Reduce Miss Rate

原理 Principle



提供更大空间容纳数据，减少容量失效与冲突失效的发生。

优点 Pros



显著降低缓存失效率，优化效果通常直观且明显。

缺点 Cons

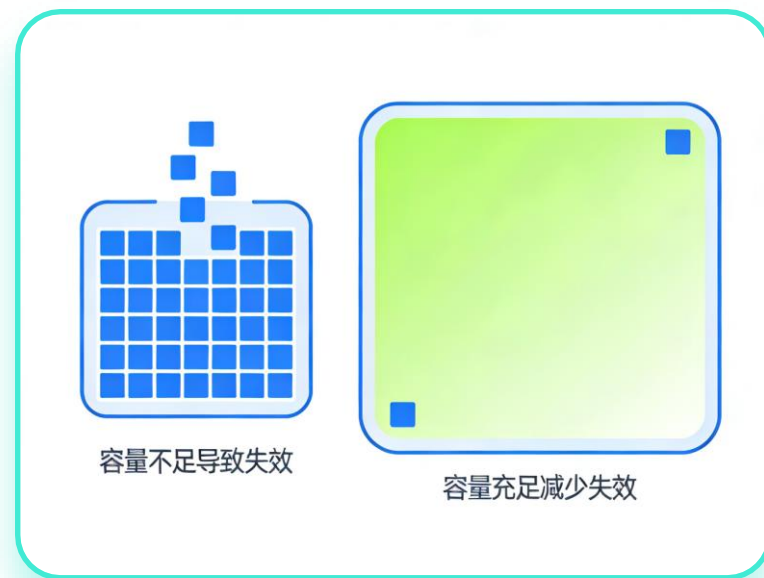


命中时间延长，同时增加硬件成本与功耗开销。

场景 Scenarios

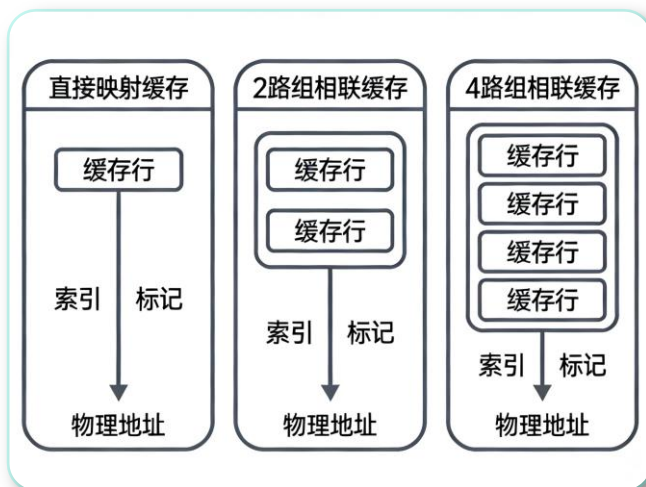


处理大型数据集的应用，如数据库查询、科学计算等。



小容量 vs 大容量缓存对比示意

3. Higher Associativity to Reduce Miss Rate



不同相联度的缓存结构对比



原理 (Principle)

允许内存块映射到缓存中的多个位置（如2路/4路组相联），而非单一固定位置，从而分散冲突风险。



优点 (Advantages)

显著降低冲突失效（Conflict Misses），在数据访问模式复杂或存在大量共享数据时效果尤为明显。



缺点 (Disadvantages)

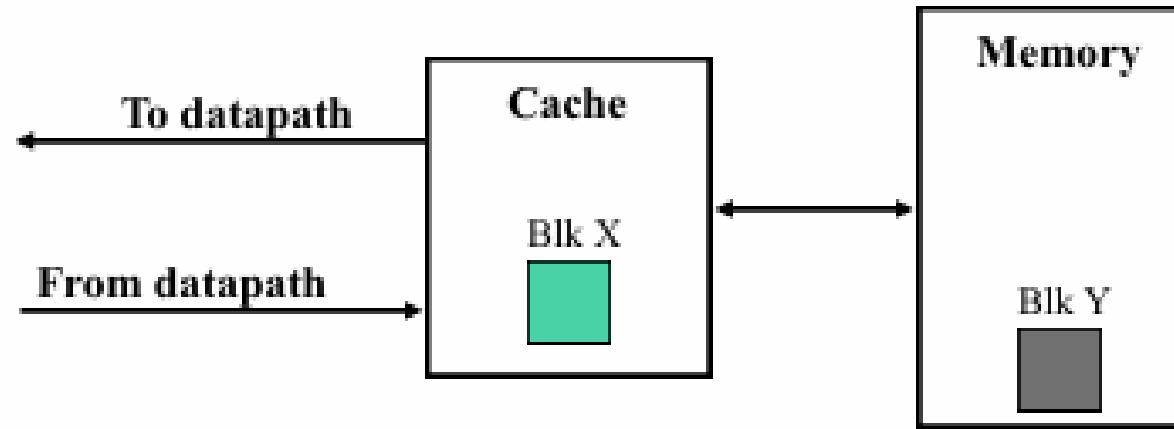
硬件复杂度提升（需并行比较多个标签），可能导致缓存命中时间（Hit Time）略有增加。



适用场景 (Scenarios)

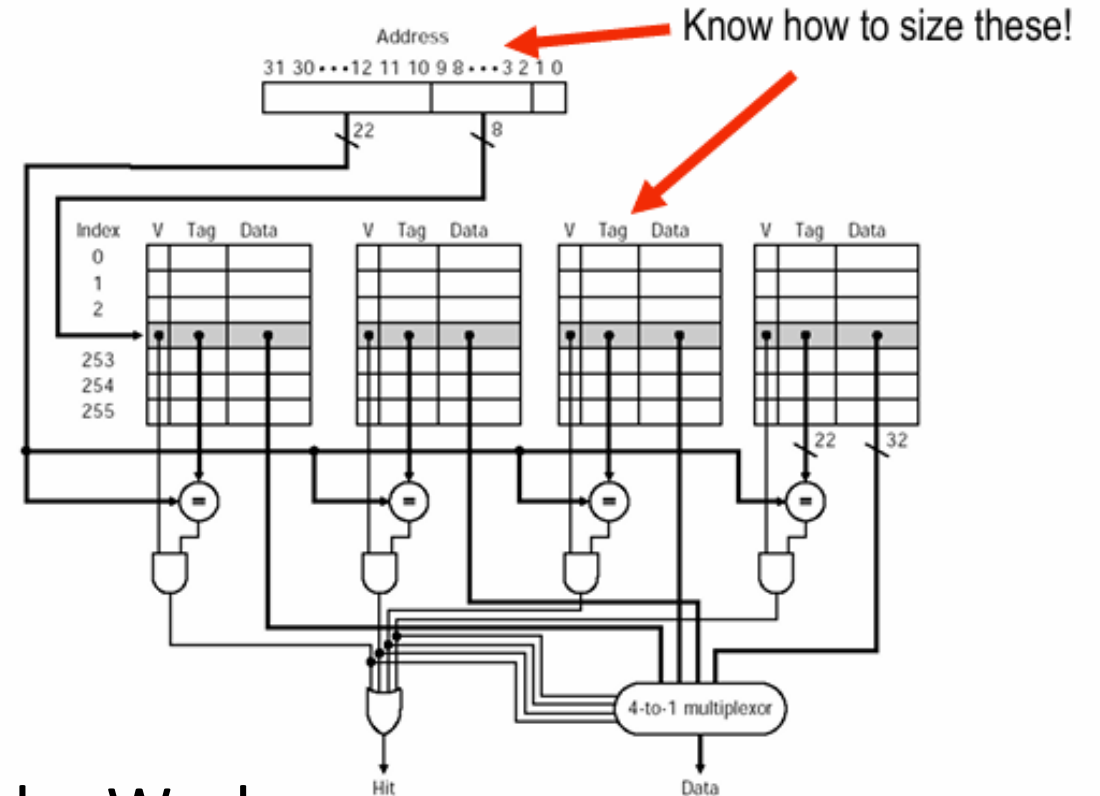
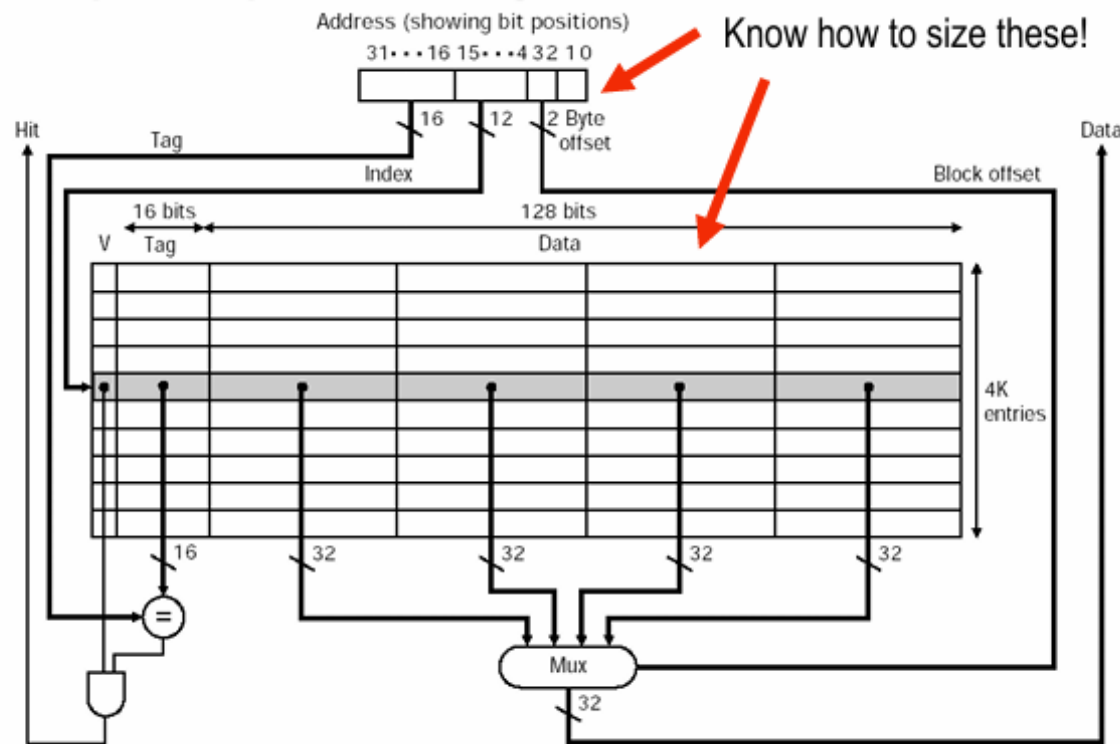
适用于数据访问模式易引发冲突失效的程序，例如数值计算、矩阵运算等科学计算类应用。

3. Higher Associativity to Reduce Miss Rate



How CPU Cache Works

3. Higher Associativity to Reduce Miss Rate



How CPU Cache Works

3. Higher Associativity to Reduce Miss Rate

□ Compulsory Miss

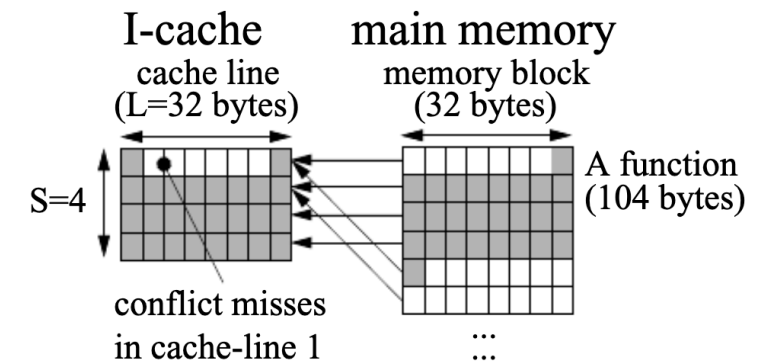
- Cold fact of life, First time data is referenced
- Run billions of instructions, become insignificant

□ Conflict Miss

- Multiple memory locations mapped to the same location
- One set fills up, but space in other cache sets
- Solution 1: increase cache size
- Solution 2: increase associative indexes

□ Capacity Miss

- Working set is larger than cache size
- Solution: increase cache size



3C Summary

3. Higher Associativity to Reduce Miss Rate

☐ 2:1 Cache Rule:

- Miss Rate of direct-mapped cache (size N) $\approx$ Miss Rate of 2-way associative cache (size $N/2$)
- Types of misses

☐ Implication:

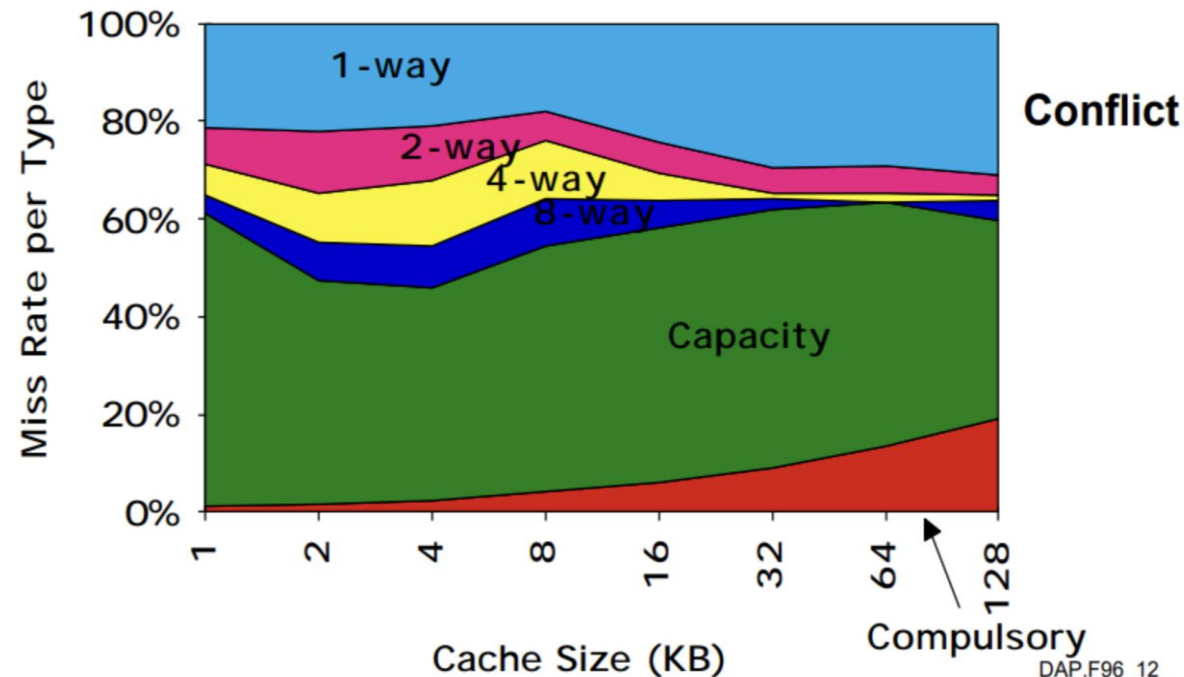
- Higher associativity reduces conflict misses.
- Helps achieve same miss rate with smaller cache.

☐ Caution – Execution Time Matters

- Miss rate is not the only metric!
- Higher associativity may increase cache hit time.
- As per Hill (1988):
 - External cache hit time increases by $\sim 10\%$
 - Internal cache by $\sim 2\%$ for 2-way vs. 1-way

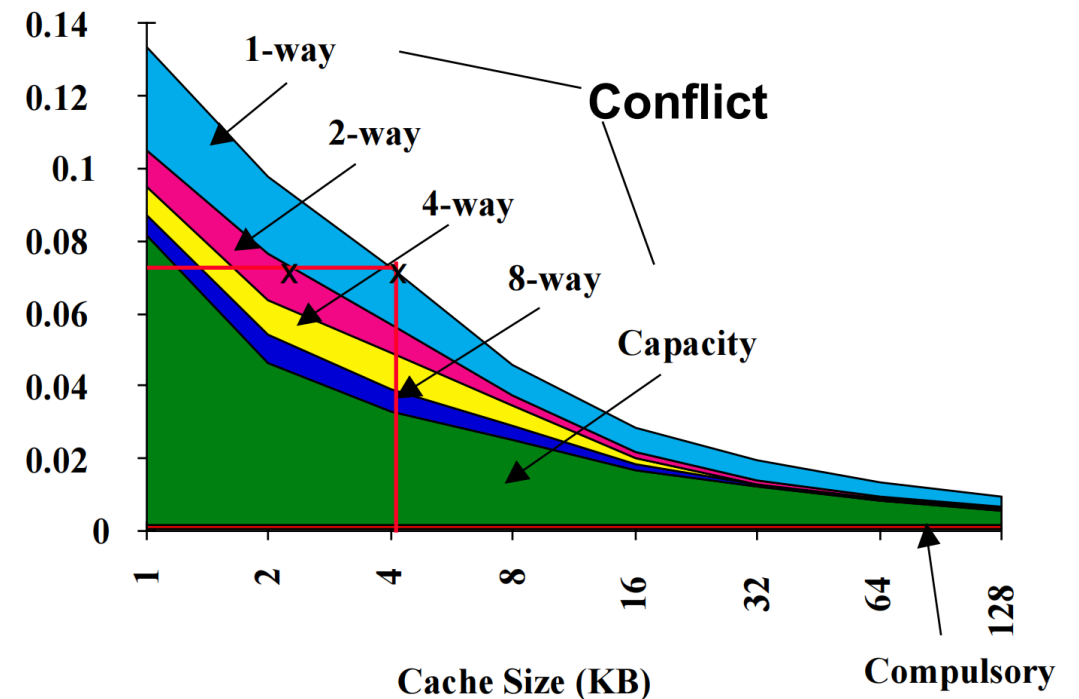
3. Higher Associativity to Reduce Miss Rate

- Relative Miss Rate



$$\text{Relative Miss Rate} = \frac{\text{Number of Misses in This Cache Level}}{\text{Number of Misses in the Higher-Level Cache}}$$

- Absolute Miss Rate



$$\text{Absolute Miss Rate} = \frac{\text{Number of Misses}}{\text{Total Number of Accesses}}$$

3. Higher Associativity to Reduce Miss Rate

- Avg. Memory Access Time vs. Miss Rate
- Assume Cache Cycle Time (CCT):
 - 1.00 (1-way), 1.10 (2-way), 1.12 (4-way), 1.14 (8-way)
 - AMAT (Access Time) across cache sizes and associativities
 - **Red cells:** No more benefit from increasing associativity

Cache Size	1-way	2-way	4-way	8-way
4	3.44	3.25	3.22	3.28
8	2.69	2.58	2.55	2.62
16	2.23	2.40	2.46	2.53
32	2.06	2.30	2.37	2.45
64	1.92	2.14	2.18	2.25
128	1.52	1.84	1.92	2.00
256	1.32	1.66	1.74	1.82
512	1.20	1.55	1.59	1.66

AVG-Memory access time = hit time + miss rate x miss penalty.

3. Higher Associativity to Reduce Miss Rate

- **Advantages:**

- Reduces conflict misses
- Better hit rate in many workloads

- **Disadvantages:**

- More complex hardware (tag comparators)
- Slightly longer access latency
- Increased power and area usage

3. Higher Associativity to Reduce Miss Rate

- Simulation Results (4KB, 8B Block Size)

cache_associativity_results_gzip.csv

	Asso,	Access Times	Load Times	Store Times	Avg Hit Rate	Load Hit Rate	Store Hit Rate
1	1,	481044,	320441,	160603,	66.6665%,	50.1927%,	99.5355%
2	2,	481044,	320441,	160603,	66.6852%,	50.2086%,	99.5598%
3	4,	481044,	320441,	160603,	66.6918%,	50.2155%,	99.566%
4	8,	481044,	320441,	160603,	66.6924%,	50.217%,	99.5648%
5	16,	481044,	320441,	160603,	66.6914%,	50.2158%,	99.5641%
6	32,	481044,	320441,	160603,	66.6914%,	50.2164%,	99.5629%
7	64,	481044,	320441,	160603,	66.6897%,	50.2149%,	99.561%
8	128,	481044,	320441,	160603,	66.6845%,	50.2105%,	99.5542%

cache_associativity_results_mcf.csv

	Asso	Access Time	Load Times	Store Time	Avg Hit Rate	Load Hit Rate	Store Hit Rate
1	1,	727230,	5972,	721258,	75.1979%,	92.063%,	75.0583%
2	2,	727230,	5972,	721258,	75.2098%,	93.3691%,	75.0594%
3	4,	727230,	5972,	721258,	75.2209%,	94.4742%,	75.0615%
4	8,	727230,	5972,	721258,	75.2227%,	94.6919%,	75.0615%
5	16,	727230,	5972,	721258,	75.2237%,	94.7756%,	75.0618%
6	32,	727230,	5972,	721258,	75.2239%,	94.7924%,	75.0619%
7	64,	727230,	5972,	721258,	75.2254%,	94.9598%,	75.062%
8	128,	727230,	5972,	721258,	75.2257%,	94.9933%,	75.062%

3. Higher Associativity to Reduce Miss Rate

- Simulation Results (4KB, 8B Block Size)

cache_associativity_results_mcf1.csv

1	Ass	Access Times	Load Times	Store Times	Avg Hit Rate	Load Hit Rate	Store Hit Rate
2	1,	727230,	5972,	721258,	0.999409%,	89.7522%,	0.264538%
3	2,	727230,	5972,	721258,	1.01206%,	90.9076%,	0.267727%
4	4,	727230,	5972,	721258,	1.0188%,	91.4099%,	0.270361%
5	8,	727230,	5972,	721258,	1.01962%,	91.6276%,	0.26939%
6	16,	727230,	5972,	721258,	1.0199%,	91.6443%,	0.269529%
7	32,	727230,	5972,	721258,	1.02127%,	91.7616%,	0.269945%
8	64,	727230,	5972,	721258,	1.02113%,	91.8453%,	0.269113%
9	128,	727230,	5972,	721258,	1.021%,	91.8285%,	0.269113%
10							

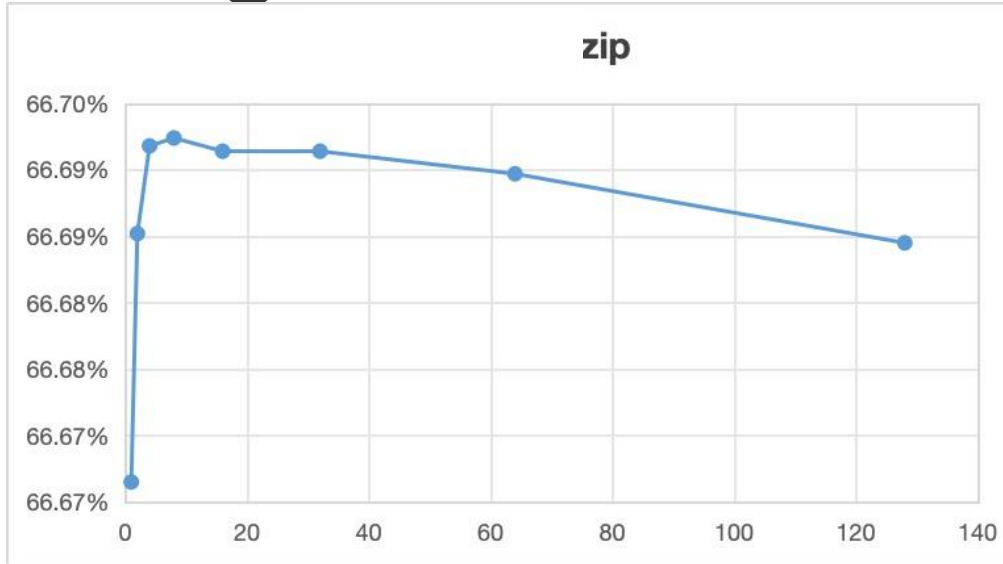
cache_associativity_results_swim.csv

1	Ass	Access Times	Load Times	Store Time	Avg Hit Rate	Load Hit Rate	Store Hit Rate
2	1,	303193,	220668,	82525,	89.6195%,	95.104%,	74.9543%
3	2,	303193,	220668,	82525,	91.3332%,	96.9166%,	76.4035%
4	4,	303193,	220668,	82525,	91.4912%,	97.0798%,	76.5477%
5	8,	303193,	220668,	82525,	91.5569%,	97.0956%,	76.7464%
6	16,	303193,	220668,	82525,	91.6842%,	97.1931%,	76.9537%
7	32,	303193,	220668,	82525,	91.7155%,	97.2157%,	77.0082%
8	64,	303193,	220668,	82525,	91.7112%,	97.2071%,	77.0154%
9	128,	303193,	220668,	82525,	91.7191%,	97.2139%,	77.0264%

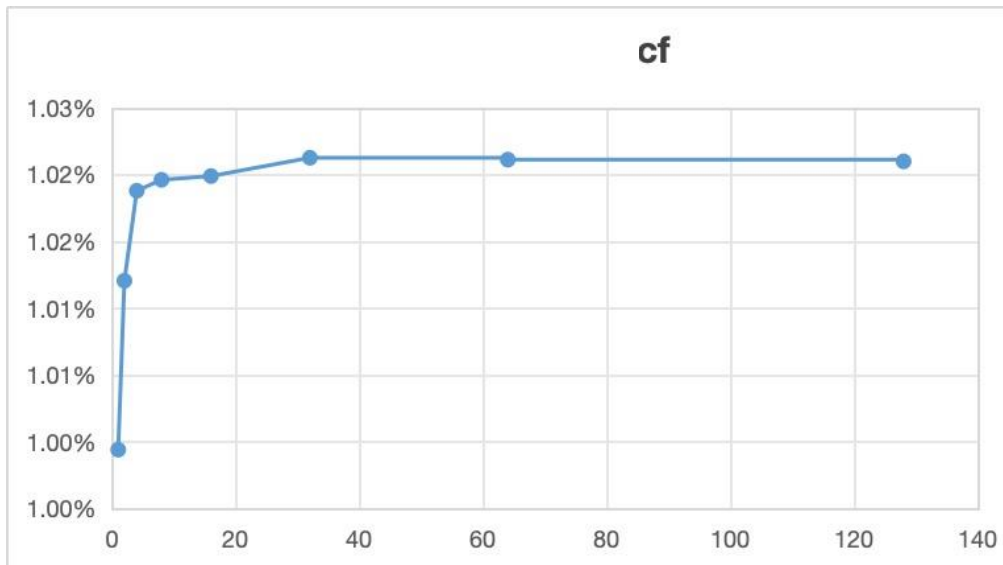
cache_associativity_results_twolf.csv

1	Ass	Access Times	Load Times	Store Times	Avg Hit Rate	Load Hit Rate	Store Hit Rate
2	1,	482824,	351403,	131421,	96.4333%,	97.1346%,	94.5579%
3	2,	482824,	351403,	131421,	98.405%,	99.2368%,	96.181%
4	4,	482824,	351403,	131421,	98.5862%,	99.4419%,	96.2982%
5	8,	482824,	351403,	131421,	98.6463%,	99.496%,	96.3743%

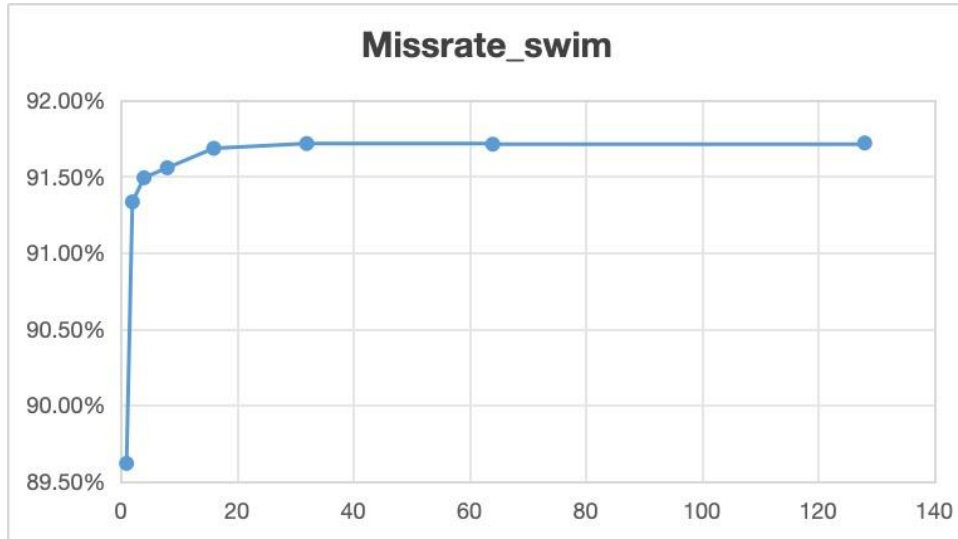
3. Higher Associativity to Reduce Miss Rate



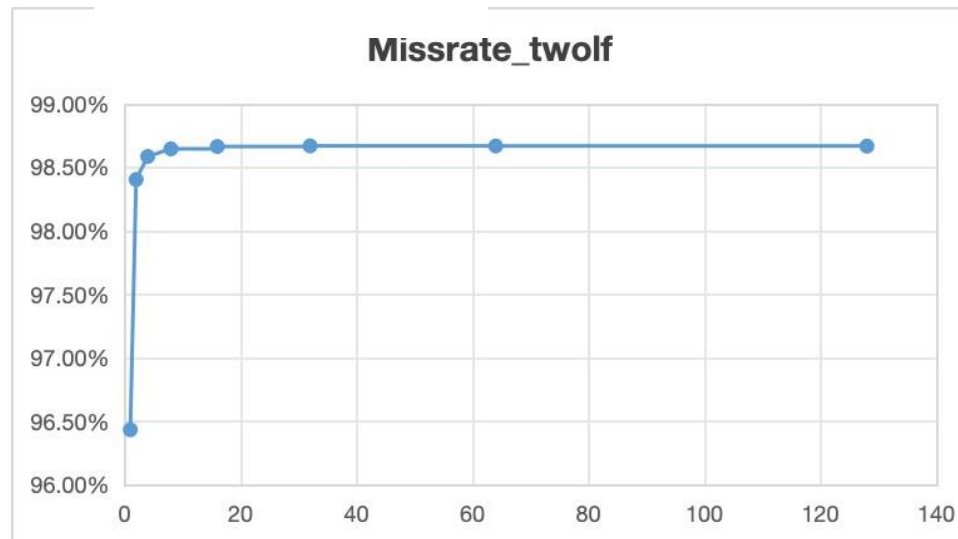
- 4-way to 8-way associativity offers a good trade-off between performance and complexity.
- **Excessively high associativity (>8-way) increases hardware cost with limited gain.**



3. Higher Associativity to Reduce Miss Rate



- 4-way to 8-way associativity offers a good trade-off between performance and complexity.
- **Excessively high associativity (>8-way) increases hardware cost with limited gain.**



3. Higher Associativity to Reduce Miss Rate

- **Real-World Cache Designs**

- Intel Core i7 (example):
 - L1: 32KB, 8-way set associative
 - L2: 256KB, 4-way
 - L3: 8MB, 16-way shared
- Design choices balance performance and cost.



- **Pseudo-Associativity**

- How to combine fast hit time of Direct Mapped and have the lower conflict misses of 2-way SA cache?
- Divide cache: on a miss, check other half of cache to see if there, if so have a **pseudo-hit** (slow hit)
- Drawback: CPU pipeline is hard if hit takes 1 or 2 cycles
 - Better for caches not tied directly to processor (L2)
 - Used in MIPS R1000 L2 cache, similar in UltraSPARC

3. Higher Associativity to Reduce Miss Rate

- Cache associativity significantly impacts miss rate.
- Set-associative caches reduce conflict misses effectively.
- Higher associativity must be balanced with hardware cost.
- Real systems typically use 2 to 8-way associativity.

4. Multilevel Caches to Reduce Miss Penalty



Core Principle

Add L1/L2/L3 caches between CPU and main memory. On L1 miss, check faster L2/L3 first to avoid slow main memory access.



Advantages

Significantly reduces miss penalty; standard in modern processors.



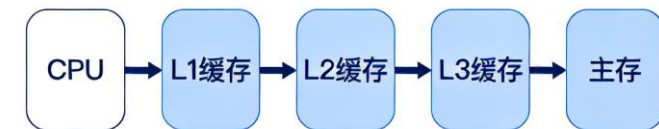
Disadvantages

Increases hardware complexity and manufacturing cost.



Application

All modern high-performance processors and computing systems.



Hierarchy of Multilevel Caches

5. Giving Priority to Read Misses over Writes to Reduce Miss Penalty

核心原理

写操作入缓冲区等待，读失效时优先处理读请求，减少CPU等待读数据的阻塞时间。

主要优点

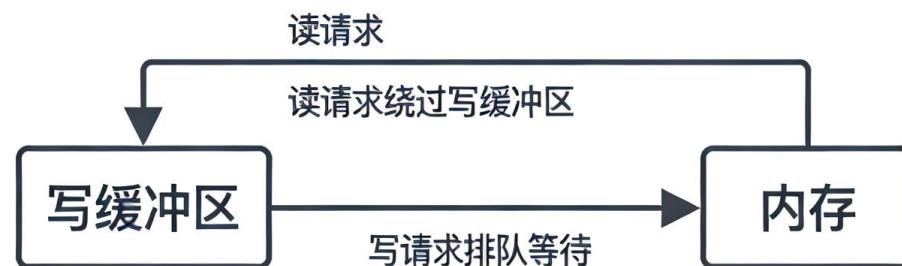
显著降低有效失效开销，在读取密集型场景下性能提升尤为明显。

潜在缺点

可能会增加写操作的延迟，但由于写操作通常对实时性不敏感，此影响较小。

适用场景

读操作频率远高于写操作的应用程序，如数据分析、搜索引擎等。



图：读请求优先处理流程示意图



01

strategies to handle read miss and write miss



浙江大學
ZHEJIANG UNIVERSITY



01 read miss strategies



浙江大学
ZHEJIANG UNIVERSITY

Read Through

不常用

Read the contents **directly from Main Memory** to the **CPU** without going through the cache.

Read Allocate

PENALTY

First read the data from the **Main Memory** to the **Cache**, and then read the data from the **Cache** to the **CPU**.



write miss strategies



浙江大学
ZHEJIANG UNIVERSITY

Write Allocate

PENALTY

Typically with
write back

First, load the data to be written from **the Main Memory** to the **Cache**, and the **CPU** writes the data to the **Cache**.

Subsequently, flush the data from the Cache to the Main Memory.

Write Around

PENALTY

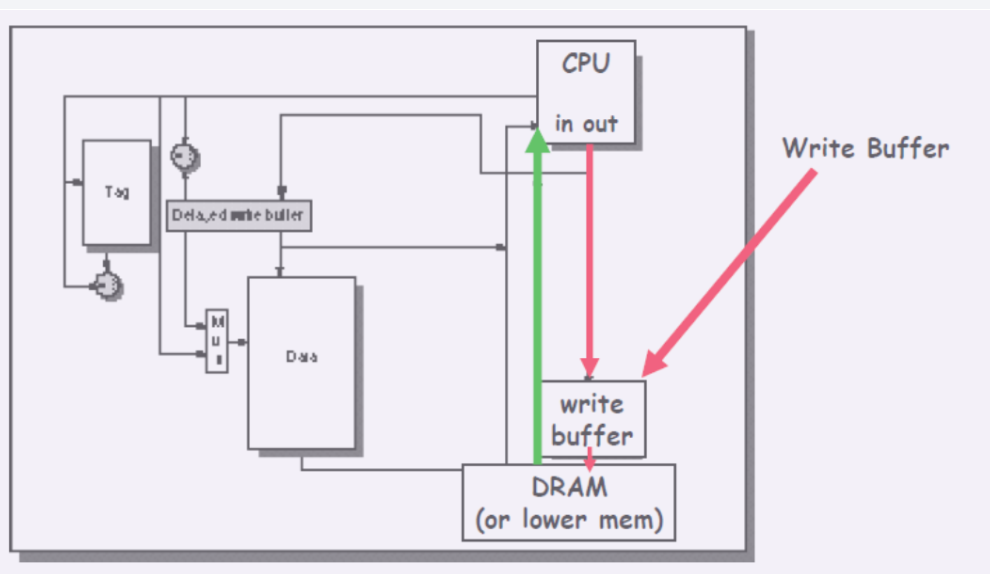
Typically with
write through

Data is modified in the Main Memory and written directly from the **CPU** to the **Main Memory**. It is not loaded into the Cache.

**Basic idea of giving reads
priority over write**



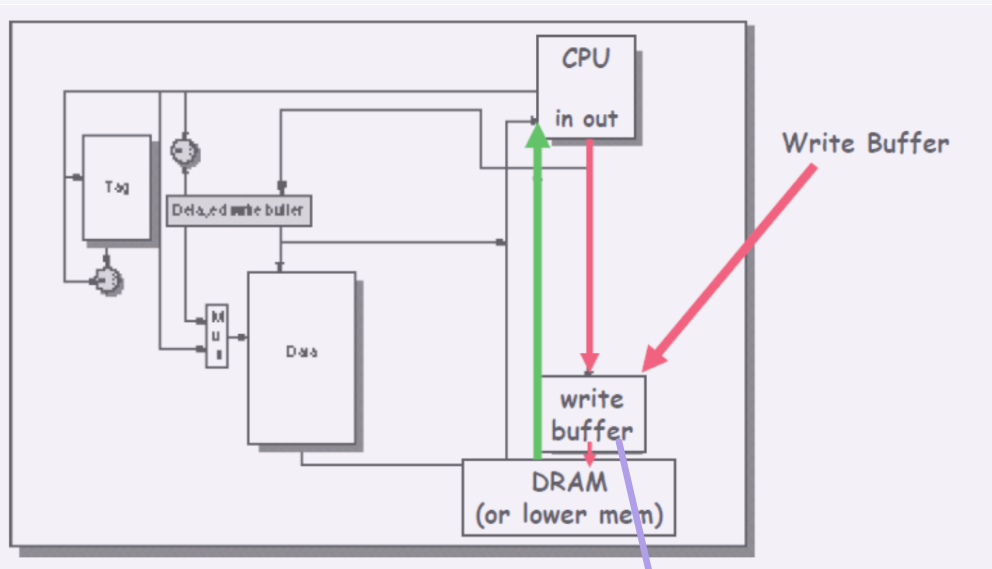
浙江大學
ZHEJIANG UNIVERSITY



- a cache consistency strategy that emphasizes in the processor's cache system
- read operations should be given higher priority for handling cache misses
- core concept: reduce CPU time waiting for data to be written to memory

RAW conflict?

When a write miss occurs...



“in the background”

Write Allocate

- first load the block into the cache for writing and set the dirty bit to 1.
- The dirty block is moved to the write buffer for asynchronous writing to memory when it is replaced.

Write Around

- directly write the content to the write buffer for asynchronous writing to memory.

03

Implementation



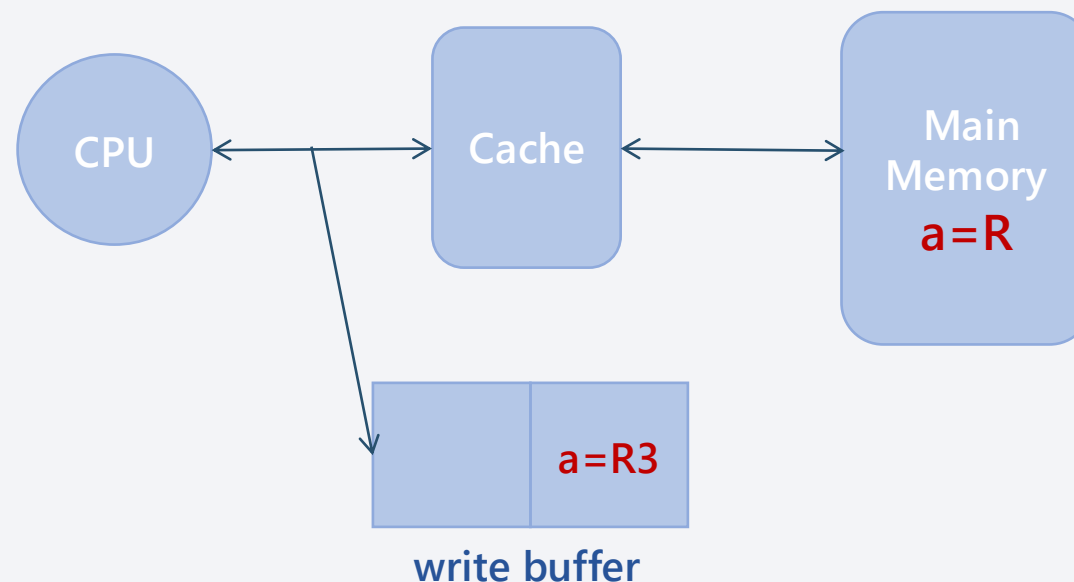
浙江大學
ZHEJIANG UNIVERSITY


```
SW R3, 512(R0) ; M[512] <- R3      (cache index 0)
LW R1, 1024(R0) ; R1 <- M[1024] (cache index 0)
LW R2, 512(R0) ; R2 <- M[512]   (cache index 0)
```

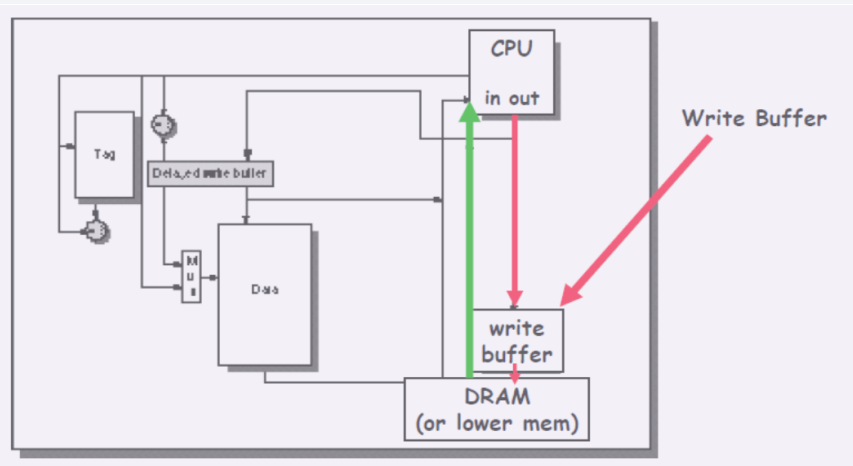
Assume a **direct-mapped** cache that maps 512 and 1024 to the same block, and a four-word write buffer that is not checked on a read miss.

Will the value in R2 always be equal to the value in R3?

RAW Conflict!!!



Read miss



➤ simplest way: read miss to **wait** until the write buffer is empty.

---- *greatly reduces CPU performance*

➤ Giving reads priority over writes: check the contents of the **write buffer** first:

- ❑ conflict happens: **retrieve data from the write buffer**

- ❑ no conflicts: let the memory access continue

---- *avoid waiting for the process of writing memory*

Performance analysis ·

04



浙江大學
ZHEJIANG UNIVERSITY

简化 CPU 的模型运行环境, 设置参数如下:

基础时间参数

- CPU 读取/写入 Cache 的时间: 1 个周期
- CPU 读取/写入 Main Memory 时间: 100 个周期
- Cache 写回/读取 Main Memory 的时间: 50 个周期
- CPU 读取/写入 Write Buffer 的时间: 3 个周期
- Cache 写入 Write Buffer 的时间: 5 个周期
- Write Buffer 写入 Main Memory 的时间: 30 个周期

访问参数

- 读操作占比: 70%
- 写操作占比: 30%
- 发生 RAW 的概率: 15%
发生 RAW 的概率
- Write Buffer Hit 率: 70%
在发生 RAW 冲突时, Data 仍在 Write Buffer 中的概率
跟写缓冲区的大小等有关
- Read Miss 率: 15%
- Write Miss 率: 10%
- 假设 Write Buffer 始终有空位



read miss



浙江大学
ZHEJIANG UNIVERSITY

简化 CPU 的模型运行环境，设置参数如下：

基础时间参数

- CPU 读取/写入 Cache 的时间：1 个周期
- CPU 读取/写入 Main Memory 时间：100 个周期
- Cache 写回/读取 Main Memory 的时间：50 个周期
- CPU 读取/写入 Write Buffer 的时间：3 个周期
- Cache 写入 Write Buffer 的时间：5 个周期
- Write Buffer 写入 Main Memory 的时间：30 个周期

访问参数

- 读操作占比：70%
- 写操作占比：30%
- 发生 RAW 的概率：15%
发生 RAW 的概率
- Write Buffer Hit 率：70%
在发生 RAW 冲突时，Data 仍在 Write Buffer 中的概率
跟写缓冲区的大小等有关
- Read Miss 率：15%
- Write Miss 率：10%
- 假设 Write Buffer 始终有空位

85% 的 read miss 没有冲突，不需要等待 Write 先写入。Cache 需要先读取 Main Memory，CPU 再读取 Cache，因此这一部分的 penalty 为：

$$\text{Penalty}(\text{no conflict}) = 50 + 1 = 51\text{cycles}$$

15% 的 read 操作存在冲突，原先需要等待引起冲突的 Write 先写入 Main Memory，再从 Main Memory 中读取 data 到 Cache 中，最后 read Cache。但“giving read priority over write” + write buffer 后，如果数据仍未写回 Main Memory，可以直接从 Write Buffer 获取到最新的数据。这一部分的 penalty 为：

- Write Buffer 异步写回 Main Memory 的执行逻辑：先写回 Main memory，再更新 Write Buffer 原位置的数据情况；也就是说在异步写回的过程中，原本需要写回的数据仍然在 Write Buffer 中

$$\text{Penalty}(\text{conflict without opt}) = 50 + 50 + 1 = 101\text{cycles}$$

$$\text{Penalty}(\text{conflict with opt}) = 3\text{cycles}$$

故 read miss penalty 为：

$$\text{Penalty}(\text{without opt}) = 51 \times 85\% + 101 \times 15\% = 58.5\text{cycles}$$

$$\text{Penalty}(\text{with opt}) = 51 \times 85\% + 3 \times 15\% = 43.8\text{cycles}$$



简化 CPU 的模型运行环境，设置参数如下：

基础时间参数

- CPU 读取/写入 Cache 的时间：1 个周期
- CPU 读取/写入 Main Memory 时间：100 个周期
- Cache 写回/读取 Main Memory 的时间：50 个周期
- CPU 读取/写入 Write Buffer 的时间：3 个周期
- Cache 写入 Write Buffer 的时间：5 个周期
- Write Buffer 写入 Main Memory 的时间：30 个周期

访问参数

- 读操作占比：70%
- 写操作占比：30%
- 发生 RAW 的概率：15%
发生 RAW 的概率
- Write Buffer Hit 率：70%
在发生 RAW 冲突时，Data 仍在 Write Buffer 中的概率
跟写缓冲区的大小等有关
- Read Miss 率：15%
- Write Miss 率：10%
- 假设 Write Buffer 始终有空位

Write Around

原先的 Write Around 会直接写入 Main Memory，此时的 penalty 为：

$$\text{Penalty}(\text{without opt}) = 100\text{cycles}$$

现在由于 Write Buffer 的存在，CPU 只需要写入 Write Buffer 即可，而 Write Buffer 写回 Main Memory 的过程则会异步处理掉，此时的 penalty 为：

$$\text{Penalty}(\text{with opt}) = 3\text{cycles}$$

Write Allocate

原先的 Write Allocate 会从 Main Memory 加载要写入的 data 到 Cache 中，再写入 Cache，在之后被替换掉的时候，再写回 Main Memory，此时的 penalty 为：

$$\text{Penalty}(\text{without opt}) = 50 + 1 + 50 = 101\text{cycles}$$

现在，在被替换掉的时候，可以写到 Write Buffer 中，而 Write Buffer 写回 Main Memory 则会异步处理，此时的 penalty 为：

$$\text{Penalty}(\text{with opt}) = 50 + 1 + 5 = 56\text{cycles}$$



performance analysis



浙江大学
ZHEJIANG UNIVERSITY

简化 CPU 的模型运行环境，设置参数如下：

基础时间参数

- CPU 读取/写入 Cache 的时间：1 个周期
- CPU 读取/写入 Main Memory 时间：100 个周期
- Cache 写回/读取 Main Memory 的时间：50 个周期
- CPU 读取/写入 Write Buffer 的时间：3 个周期
- Cache 写入 Write Buffer 的时间：5 个周期
- Write Buffer 写入 Main Memory 的时间：30 个周期

访问参数

- 读操作占比：70%
- 写操作占比：30%
- 发生 RAW 的概率：15%
发生 RAW 的概率
- Write Buffer Hit 率：70%
在发生 RAW 冲突时，Data 仍在 Write Buffer 中的概率
跟写缓冲区的大小等有关
- Read Miss 率：15%
- Write Miss 率：10%
- 假设 Write Buffer 始终有空位

综上，得到整体的 Penalty 变化如下：

对于 Write Around：

$$Penalty(\text{without opt}) = 100 \times 30\% + 58.5 \times 70\% = 70.95 \text{cycles}$$

$$Penalty(\text{with opt}) = 3 \times 30\% + 43.8 \times 70\% = 31.56 \text{cycles}$$

$$opt_rate = \frac{\frac{1}{31.56}}{\frac{1}{70.95}} = 2.25$$

对于 Write Allocate：

$$Penalty(\text{without opt}) = 101 \times 30\% + 58.5 \times 70\% = 71.25 \text{cycles}$$

$$Penalty(\text{with opt}) = 56 \times 30\% + 43.8 \times 70\% = 47.46 \text{cycles}$$

$$opt_rate = \frac{\frac{1}{47.46}}{\frac{1}{71.25}} = 1.50$$



performance analysis



浙江大学
ZHEJIANG UNIVERSITY

简化 CPU 的模型运行环境，设置参数如下：

基础时间参数

- CPU 读取/写入 Cache 的时间：1 个周期
- CPU 读取/写入 Main Memory 时间：100 个周期
- Cache 写回/读取 Main Memory 的时间：50 个周期
- CPU 读取/写入 Write Buffer 的时间：3 个周期
- Cache 写入 Write Buffer 的时间：5 个周期
- Write Buffer 写入 Main Memory 的时间：30 个周期

访问参数

- 读操作占比：70%
- 写操作占比：30%
- 发生 RAW 的概率：15%
发生 RAW 的概率
- Write Buffer Hit 率：70%
在发生 RAW 冲突时，Data 仍在 Write Buffer 中的概率
跟写缓冲区的大小等有关
- Read Miss 率：15%
- Write Miss 率：10%
- 假设 Write Buffer 始终有空位

而优化后的 Penalty，对整体程序的性能的影响如下：

对 Write Around：

$$Total(\text{without opt}) = 58.5 \times 15\% + 70.95 \times 10\% + 1 \times 75\% = 16.62 \text{cycles}$$

$$Total(\text{with opt}) = 43.8 \times 15\% + 31.56 \times 10\% + 1 \times 75\% = 10.48 \text{cycles}$$

$$opt_rate = \frac{\frac{1}{10.48}}{\frac{1}{16.62}} = 1.59$$

对 Write Allocate

$$Total(\text{without opt}) = 58.5 \times 15\% + 71.25 \times 10\% + 1 \times 75\% = 16.65 \text{cycles}$$

$$Total(\text{with opt}) = 43.8 \times 15\% + 47.46 \times 10\% + 1 \times 75\% = 12.07 \text{cycles}$$

$$opt_rate = \frac{\frac{1}{12.07}}{\frac{1}{16.65}} = 1.38$$

6. Avoiding Address Translation During Indexing of the Cache to Reduce Hit Time



Core Principle

Use virtual addresses for cache indexing instead of physical addresses, bypassing TLB lookups on the critical path.



Advantages

Significantly reduces cache hit time and allows for higher CPU clock frequencies due to reduced latency.



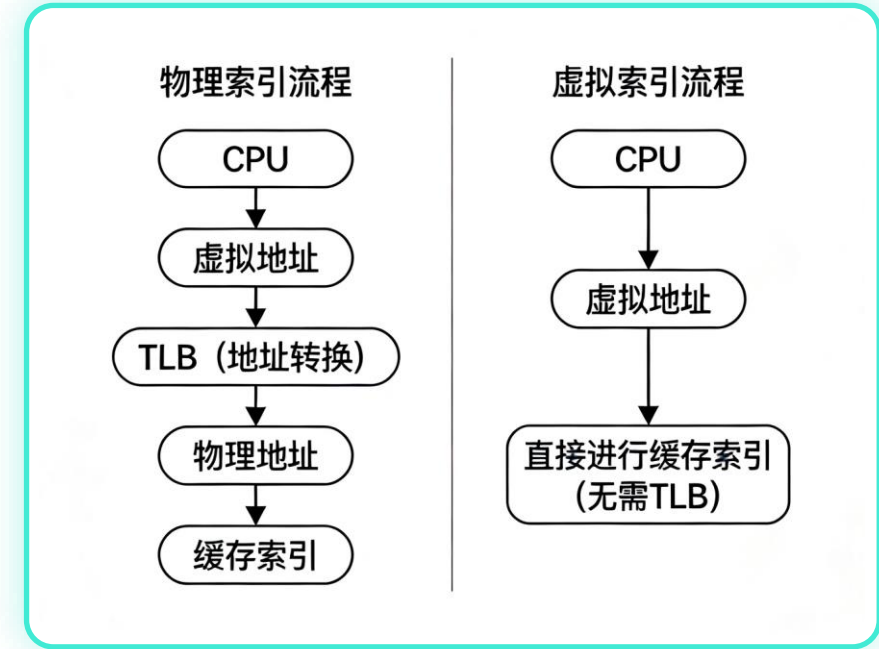
Drawbacks

Risk of cache aliasing (multiple virtual addresses mapping to same physical address), requiring special hardware solutions.

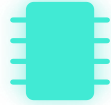


Use Cases

High-performance processor designs where every cycle of latency must be minimized for peak efficiency.



Virtual vs. Physical Indexing Flow Comparison



03. Advanced Optimization Methods

Advanced Techniques & Compiler Support

7. Pipelined L1 Caches With Virtual Indexing and Set Associativity



核心原理

将缓存访问（译码、比较、读取）拆分为流水线阶段，允许CPU每个周期发起新请求，提升带宽。



主要优势

显著增加缓存带宽，支持更高的CPU时钟频率，提升连续访问的吞吐量。



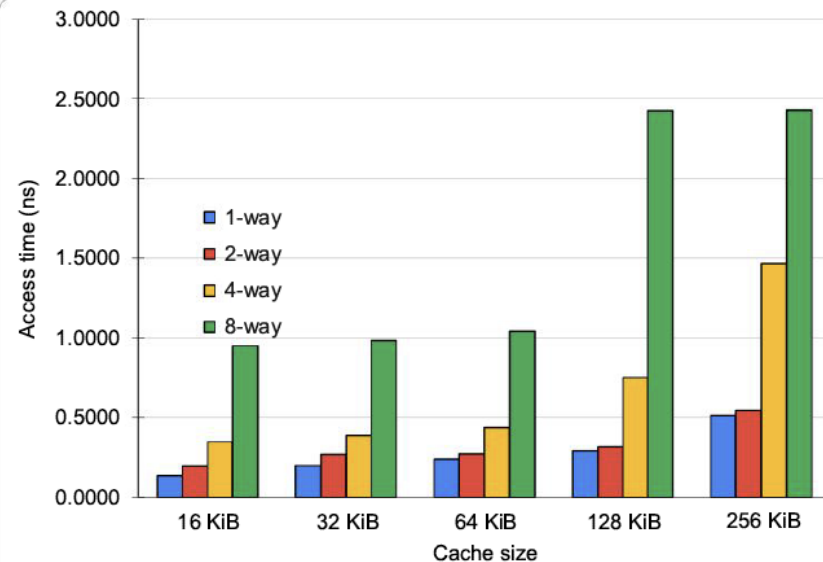
潜在代价

增加了单次访问的命中时间（延迟），数据需流经多个流水线阶段才能返回。



适用场景

适用于高频率、高带宽需求的现代高性能处理器架构。



Estimated access times generally increase as cache size and associativity are increased

7. Pipelined L1 Caches With Virtual Indexing and Set Associativity

Example: Using the data in Table in P8 and Figure in P15, determine whether a 32 KiB four-way set associative L1 cache has a faster memory access time than a 32 KiB two-way set associative L1 cache. Assume the miss penalty to L2 is 15 times the access time for the faster L1 cache. Ignore misses beyond L2. Which has the faster average memory access time?

Answer:

Let the access time for the two-way set associative cache be 1.

Then, *for the two way cache*,

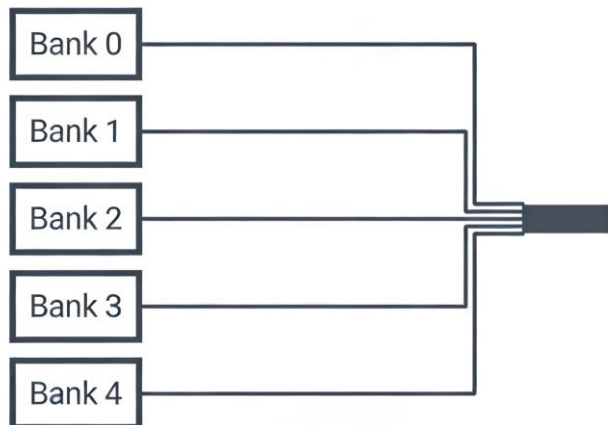
$$\begin{aligned}\text{Average memory access time}_{2\text{ way}} &= \text{Hit time} + \text{Miss rate} \times \text{Miss penalty} \\ &= 1 + 0.038 \times 15 = 1.38\end{aligned}$$

For the four-way cache,

the access time is 1.4 times longer. The elapsed time of the miss penalty is $15/1.4 = 10.1$. Assume 10 for simplicity:

$$\begin{aligned}\text{Average memory access time}_{4\text{ way}} &= \text{Hit time}_{2\text{ way}} \times 1.4 + \text{Miss rate} \times \text{Miss penalty} \\ &= 1.4 + 0.037 \times 10 = 1.77\end{aligned}$$

8. Multiple Banks and Ports to Increase the Bandwidth of L1 D-Caches



缓存多体并行访问结构示意图



核心原理：将缓存划分为多个独立存储体(Banks)，各Bank拥有独立读写端口，支持CPU并行访问以提升总带宽。



技术优势：显著增加缓存带宽，完美适配多核处理器或乱序执行CPU的高并发数据访问需求。

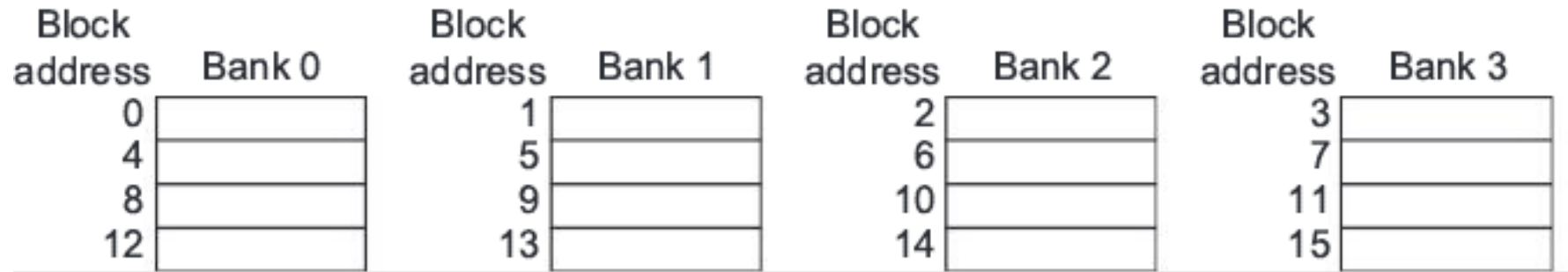


潜在挑战：引入了更复杂的硬件控制逻辑，增加了芯片设计与验证的复杂度。

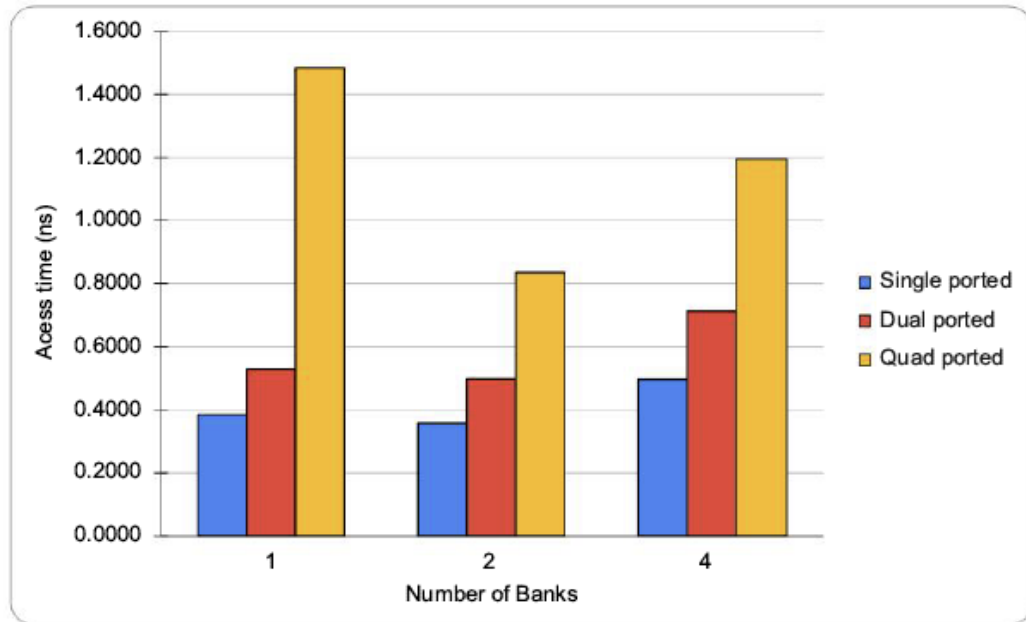


适用场景：高性能多核处理器架构、图形处理单元(GPU)及其他需要极高内存带宽的应用。

8. Multiple Banks and Ports to Increase the Bandwidth of L1 D-Caches



Four-way interleaved cache banks using block addressing. Assuming 64 bytes per block, each of these addresses would be multiplied by 64 to get byte addressing.



CACTI estimated access times for a 32 KiB, 4-way set associative cache with 32 B blocks varying the number of banks and ports.

9. Reducing the Miss Rate With Better Replacement Policies

核心原理 (Mechanism)

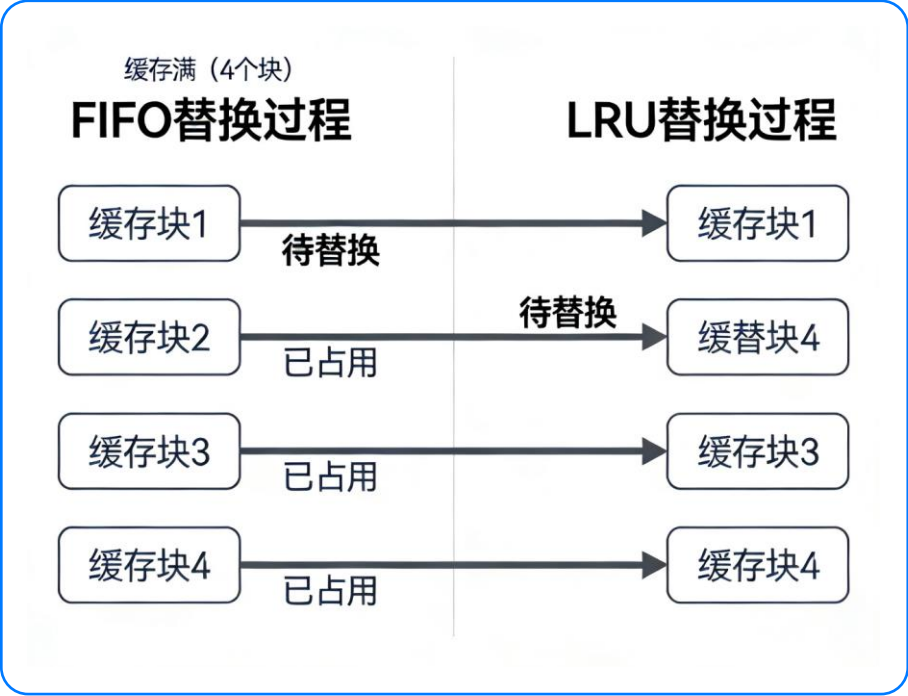
使用LRU（最近最少使用）或LFU（最不经常使用）等智能算法预测未来访问模式，优先替换无用数据，优于简单的FIFO策略。

优缺点 (Pros & Cons)

- 优点：显著降低缓存失效率。
- 缺点：增加硬件复杂度与实现开销。

适用场景 (Scenarios)

缓存容量受限，且数据访问模式具有较强局部性与可预测性的场景。



图示：FIFO与LRU缓存替换策略对比

10. Multibanked L2 and L3 Caches to Decrease Power and Latency, and Increase Bandwidth

核心原理

将大容量L2/L3缓存划分为多个独立Bank。仅激活被访问的Bank，增加带宽的同时降低功耗与延迟。

优势分析

显著降低功耗，大幅提高缓存带宽，减少访问延迟。

潜在挑战

多Bank的管理逻辑增加了硬件设计的复杂度。



适用：现代高性能多核处理器的L2/L3缓存设计。

Multibanked Cache Architecture

Processing Cores (L1 Private)



Shared L3 Cache (Partitioned Banks)



Each request accesses only one bank, reducing power consumption.

11. Nonblocking Caches to Increase Cache Bandwidth



核心原理

允许CPU在处理缓存失效时继续执行后续指令，计算与失效处理重叠，降低开销。



主要优点

显著降低有效失效开销，充分利用CPU的乱序执行能力，提升整体吞吐量。



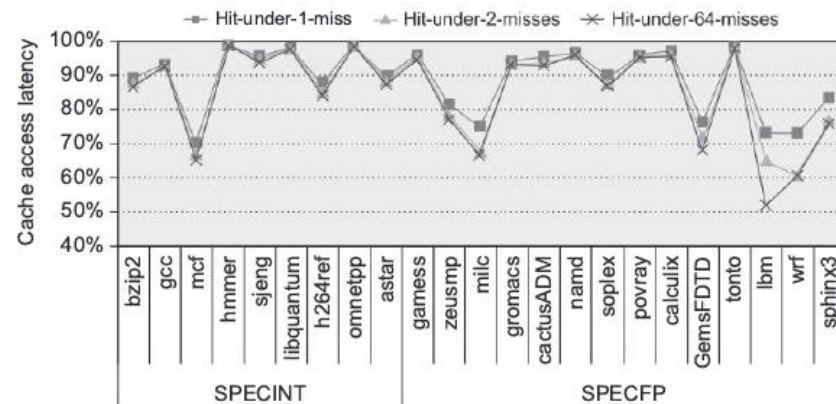
潜在缺点

大大增加了缓存控制器和CPU的硬件设计复杂度，带来更高的制造成本。



适用场景

适用于追求极致性能、支持乱序执行（Out-of-Order）的高端处理器架构。

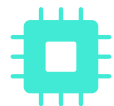


The effectiveness of a nonblocking cache is evaluated by allowing 1, 2, or 64 hits under a cache miss with 9 SPECINT (on the left) and 9 SPECFP (on the right) benchmarks.

12. Critical Word First and Early Restart to Reduce Miss Penalty

工作原理 (Mechanism)

当缓存失效时，内存控制器优先将CPU急需的**关键数据字 (Critical Word)**发送给CPU。CPU收到后立即恢复执行，无需等待整个数据块传输完成。



CPU



Main
Memory

Early Restart:

优先发送关键字，
减少等待延迟



显著降低有效失效开销

通过减少CPU的无效等待时间，提升整体执行效率。



增加硬件复杂度

内存控制器需要更复杂的逻辑来处理非连续的数据请求和乱序传输。



高延迟敏感型应用

适用于对访存延迟极其敏感，且经常访问大块数据中随机位置的程序。

核心价值：以硬件复杂度的提升换取CPU计算资源的高效利用

13. Compiler Optimizations to Reduce Miss Rate

核心原理

优化代码结构与数据访问模式，提升时间与空间局部性，减少缓存失效。常见手段包括循环交换、循环分块等。

主要优点

纯软件层面优化，无需修改硬件设施，实施成本低，易于部署和维护。

潜在局限

效果高度依赖编译器的智能优化能力，同时也受限于程序员编写的原始代码结构质量。

适用场景

适用于各类应用程序，尤其在科学计算、大数据分析等数据密集型程序中效果显著。

目录 / CONTENTS



Instruction Miss

- Reorder procedures in memory to reduce conflict misses
- Profiling to look at conflicts



Data Miss

- Merging arrays
- Loop Interchange
- Loop Fusion
- Loop Distribution
- Blocking



PART ONE

Instruction

Miss



Reorder procedures in memory to reduce conflict misses

将连续调用的函数相邻存放，存到同一个Cache line（对应内存的一个block）中，提升空间局部性。

将条件分支的Hot Path连续布局，减少跳转导致的缓存行替换。即在存在分支情况下，Hot Path这条路径极大概率是连续调用的，那么就要相邻存放。这涉及到编译器的冷热代码分离技术。

```
// 未优化布局（冷热混杂）
if (condition) { // 分支预测热路径
    hot_code(); // 地址0x100
} else {
    cold_code(); // 地址0x120 → 可能触发缓存行替换
}
more_hot();
```

```
// 优化后布局
if (condition) {
    hot_code(); // 地址0x100
} else {
    // 冷代码被编译器重定位到远端
    cold_code(); // 地址0xC000
}
more_hot(); // 地址0x110 → 同缓存行
```

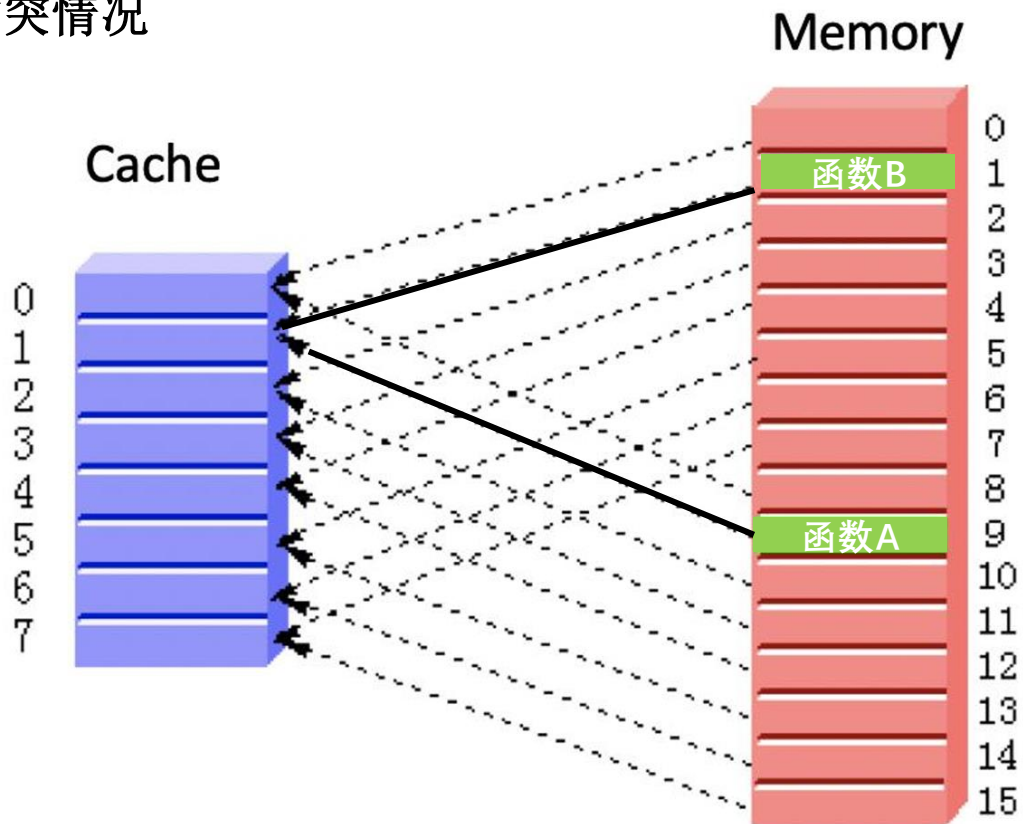


Reorder procedures in memory to reduce conflict misses

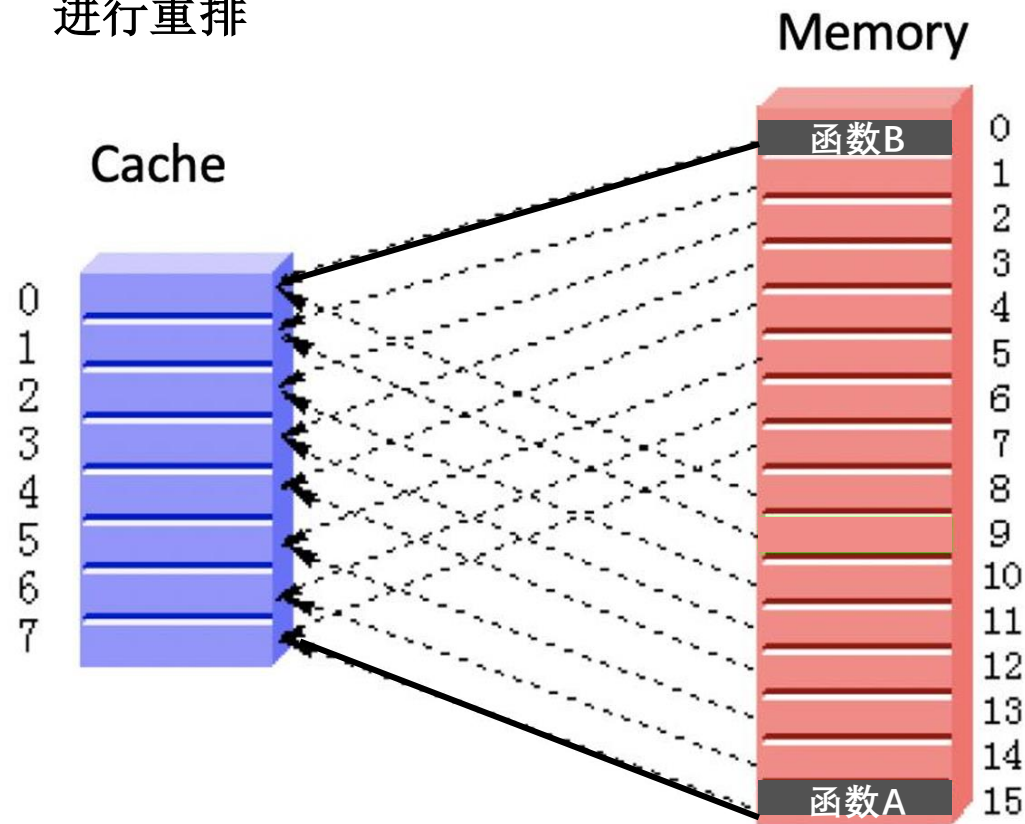


将高频调用的函数存放在不同缓存组，减少冲突。如果不在一个block中，那就尽可能在不同的缓存组 (Cache Set) 中，避免高频调用的函数的缓存行相互驱逐。

冲突情况



进行重排





Profiling to look at conflicts



动态分析 (profiling): 收集执行轨迹, 记录程序运行的指令访问模式、缓存未命中事件等

冲突检测 (Conflict Detection): 通过时间关系图, 分析代码段交替访问模式, 标记高频交替访问且映射到同一缓存组的地址对。

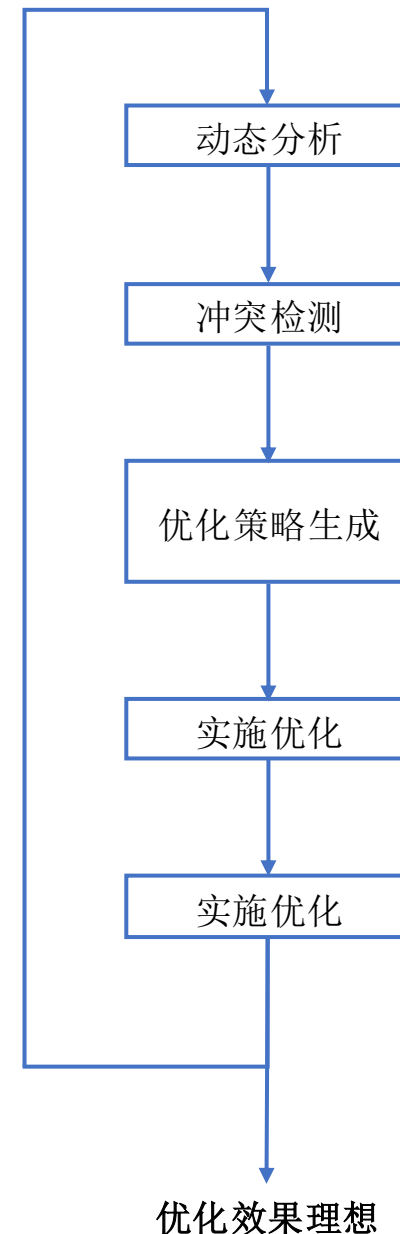
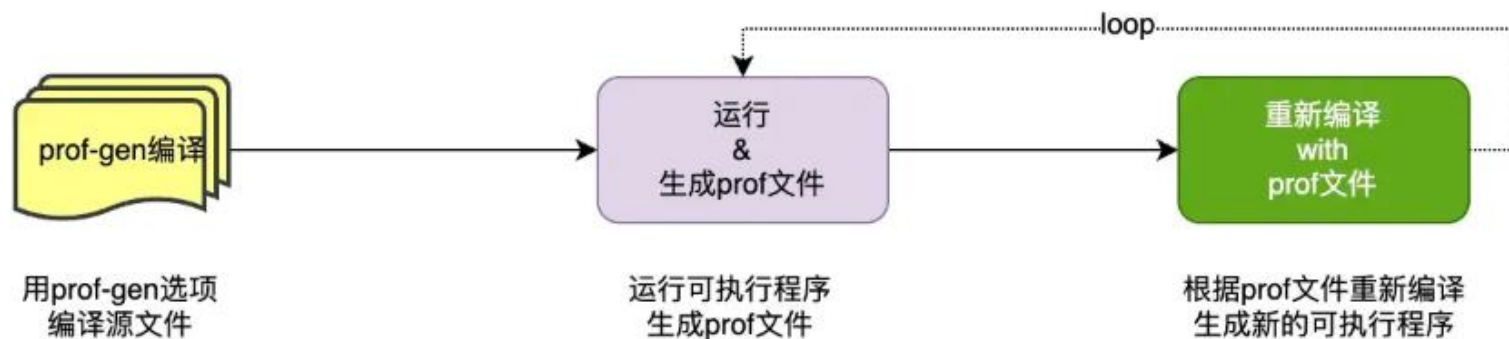
优化策略生成 (Optimization Strategy):

- 对连续调用链函数, 启用函数重排序或内联 (将函数的代码直接插入到每个调用函数位置)
- 对条件分支的 Hot Path, 基于分支概率重排基本块, 冷热代码分离, 隔离低频代码;
- 高频函数/循环体地址对齐, 并分散到不同缓存组;

实施优化 (Implementation) 可能有链接脚本控制调整关键函数地址等手动干预

验证反馈 (Validation & Feedback) 优化效果不佳则重启流程

优化效果不佳





Data Miss

PART TWO



Merging Arrays

- 假设需要读取val[]、key[]的值，若作为两个array存放，二者之间交替访问会造成大量的cache miss；而融合之后，就避免了内存的交替访问，增加了命中率。
- 利用空间相邻方便数据读取，将同时要读的数据在内存中放在一起，可以节省内存访问。

Before:								
int val[]	ADDRESS #666	val_0	val_1	val_2				
int key[]	ADDRESS #888	key_0	key_1	key_2				
After:								
struct merge dat[]	ADDRESS #555	key_0	val_0	key_1	val_1	key_2	val_2	

图1: merging arrays



Loop Interchange



change nesting of loops (storage methods of data in memory):

- 二维数组在内存中的存储方式为行主序，如下图。
- 如果先以col作为外部循环，再以row作为内部循环，访问顺序就是 $x[0][0]$, $x[1][0]$, $x[2][0]$... 相隔太远会导致较大的miss rate。
- 而如果先以row作为外部循环，访问顺序就是 $x[0][0]$, $x[0][1]$... 相邻者可一并读取，降低Miss Rate。

$x[0][0]$	$x[0][1]$	$x[0][2]$		$x[1][0]$	$x[1][1]$	$x[1][2]$			
Row 0				Row 1				Row 2	

图2: 二维数组的存储方式



Loop Fusion

- 合并具有相同循环且某些变量重叠的独立循环
- 独立：融合后逻辑关系不会发生改变
- 有变量重合：减少该变量内存访问次数



Loop distribution

- 将不相关 / 矛盾的操作划分为不同的循环

```
for (i = 0; i < n ; i++){  
    for (j = 0; j < n; j++){  
        val[i][j]++;  
        key[j][i]++;  
    }  
}
```

- 上述代码块中，val与key均希望能以row作为外部循环，但在此处会产生冲突，因此需要将二者分割，以得到最低的cache miss rate



Blocking

循环分块是指通过增加循环嵌套的维度来提升数据局部性的循环变换技术，是对多重循环的迭代空间进行重新划分的过程。

右以矩阵的乘法为例：

```
/* Before */
for (i = 0; i < N; i = i+1)
    for (j = 0; j < N; j = j+1){
        r = 0;
        for (k = 0; k < N; k = k+1)
            r = r + y[i][k]*z[k][j]; // 关键点: z[k][j]的访问
        x[i][j] = r;
    }
```

```
/* After */
for (int kk = 0; kk < n; kk += B) {
    int k_end = min(kk + B, n);
    for (int ii = 0; ii < n; ii += B) {
        int i_end = min(ii + B, n);
        for (int jj = 0; jj < n; jj += B) {
            int j_end = min(jj + B, n);
            // 上面三个循环用于遍历分块
            // 处理分块内部计算
            for (int k = kk; k < k_end; ++k) {
                for (int i = ii; i < i_end; ++i) {
                    float yval = y[i * n + k];
                    for (int j = jj; j < j_end; ++j) {
                        x[i * n + j] += yval * z[k * n + j];
                    }
                }
            }
        }
    }
}
```



Blocking



浙江大学
ZHEJIANG UNIVERSITY

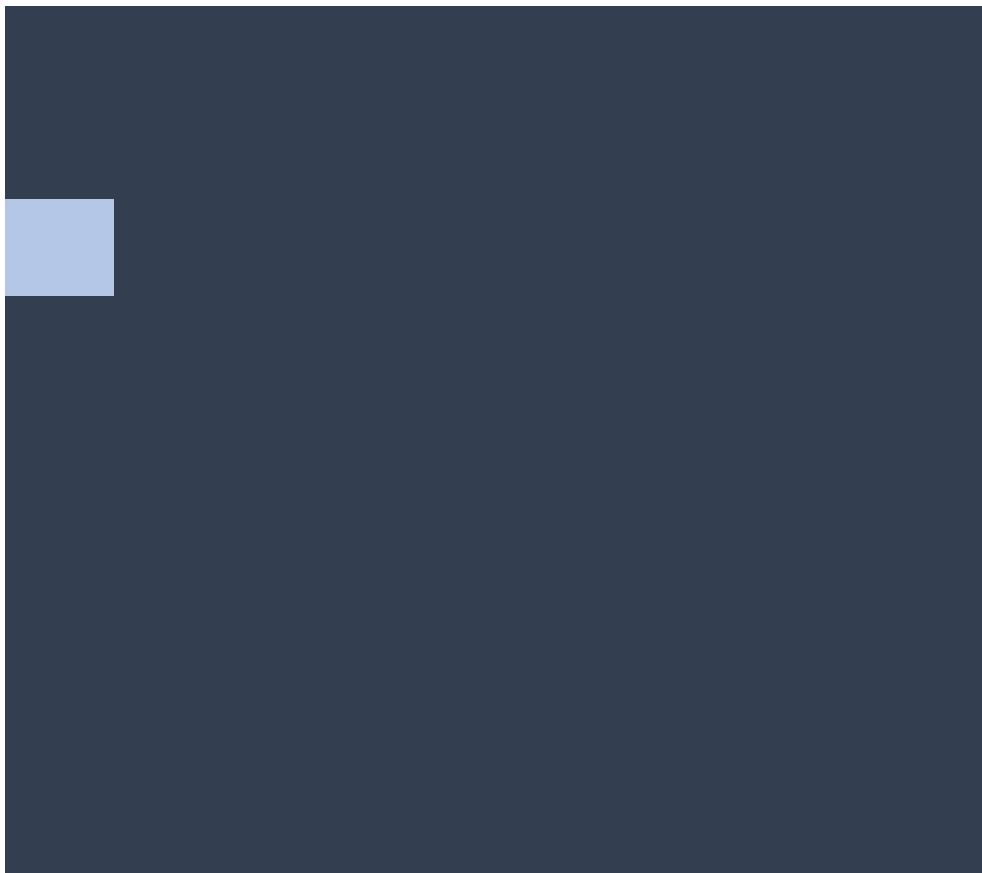




Blocking



浙江大学
ZHEJIANG UNIVERSITY

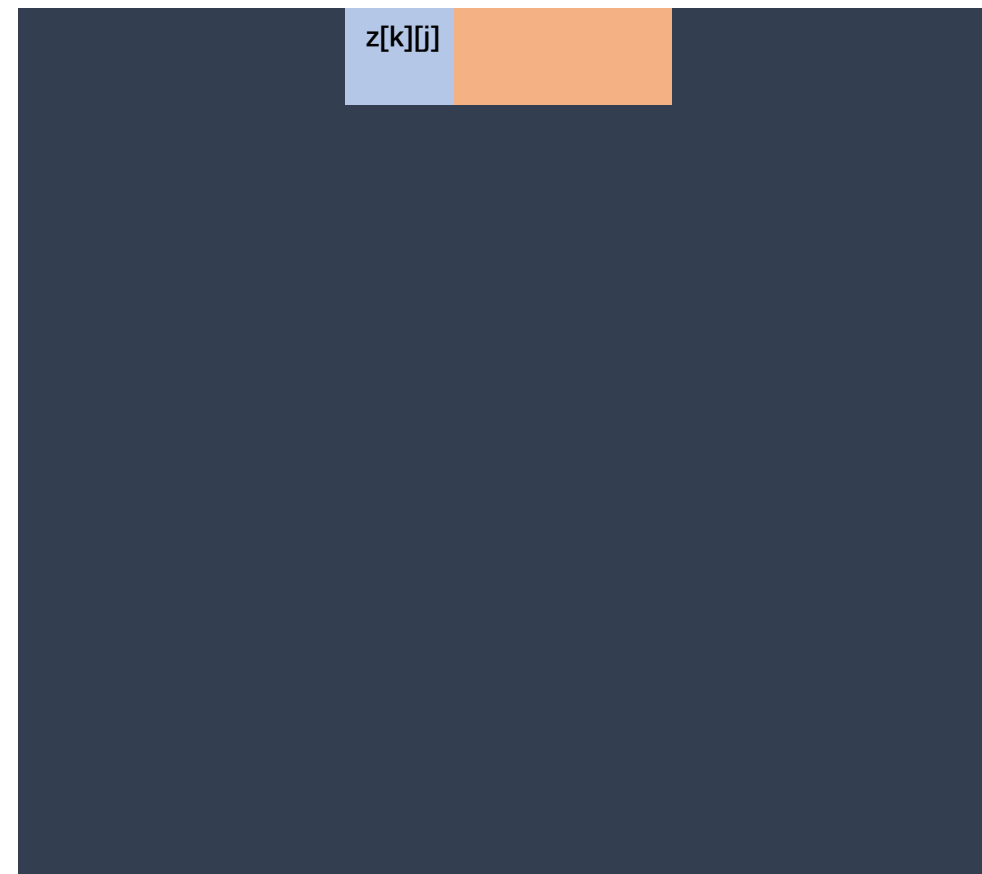
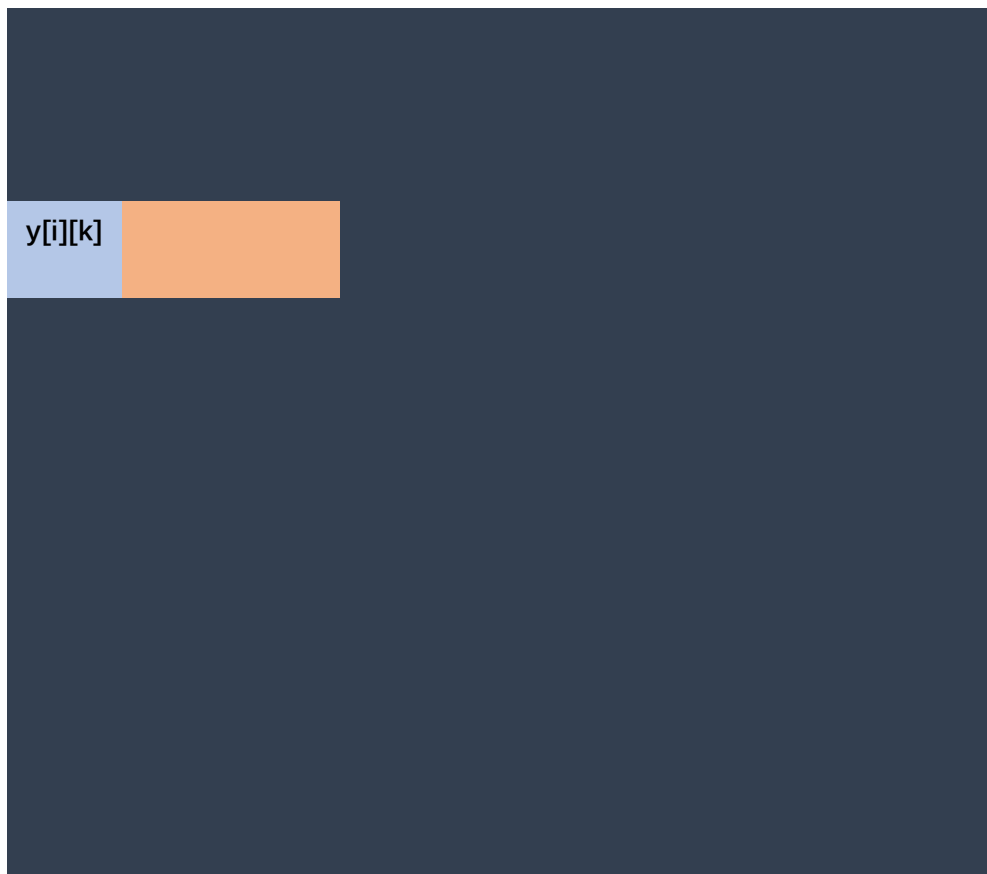




Blocking



```
for (k = 0; k < N; k = k+1)  
  r = r + y[i][k]*z[k][j]; // 关键点: z[k][j]的访问
```

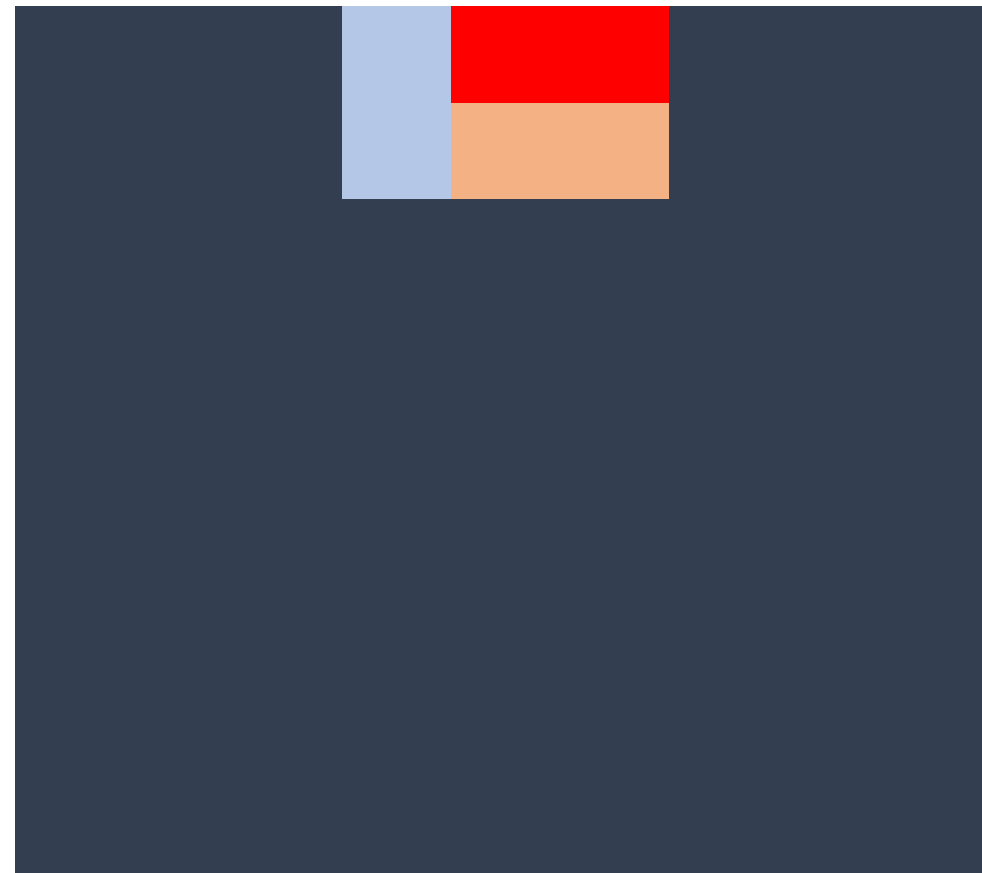
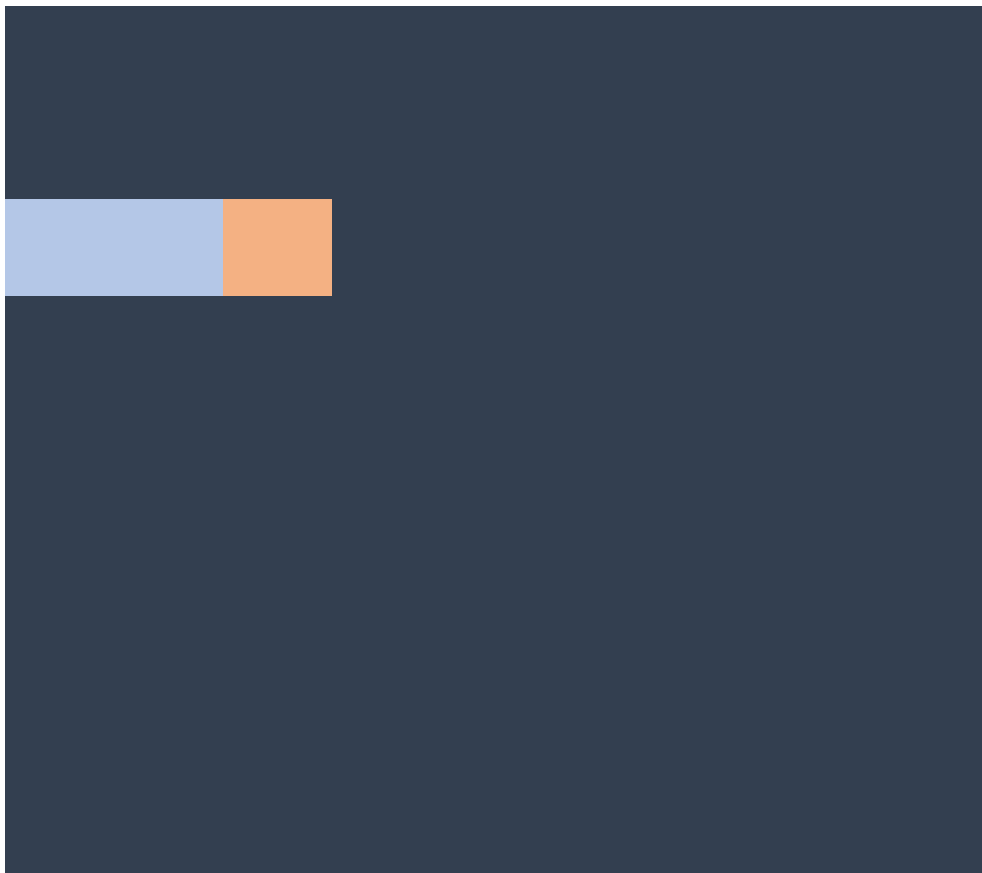




Blocking



浙江大学
ZHEJIANG UNIVERSITY

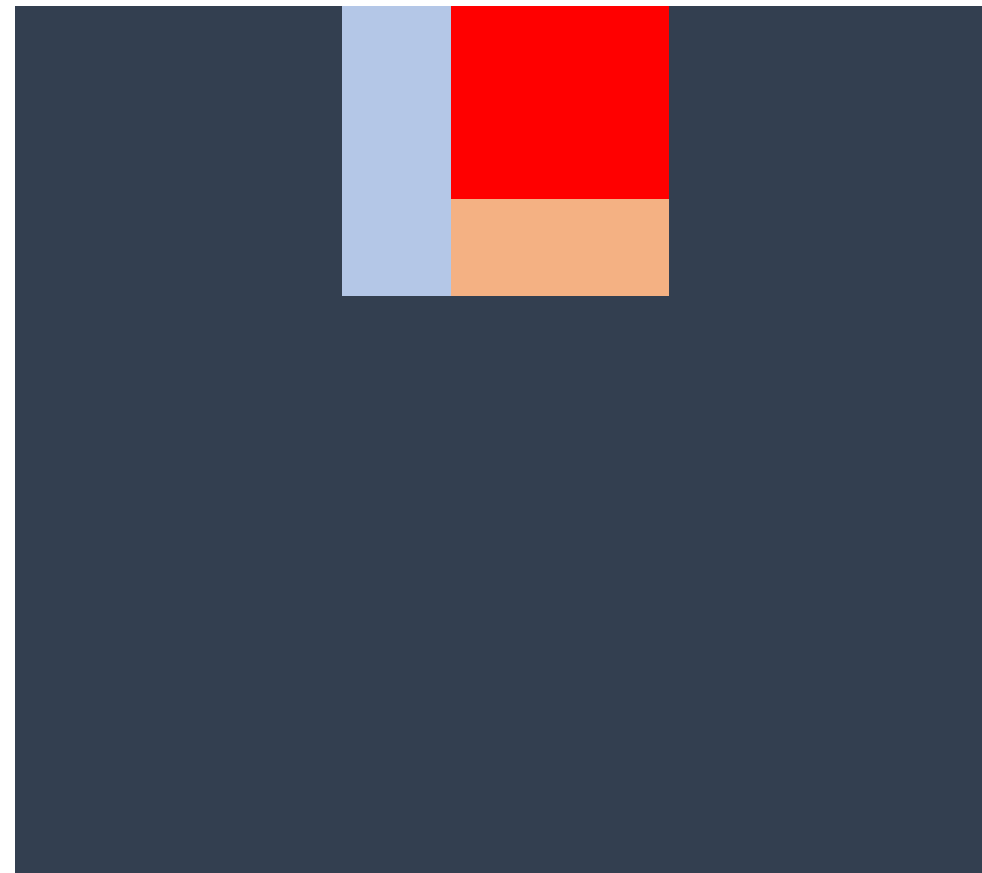
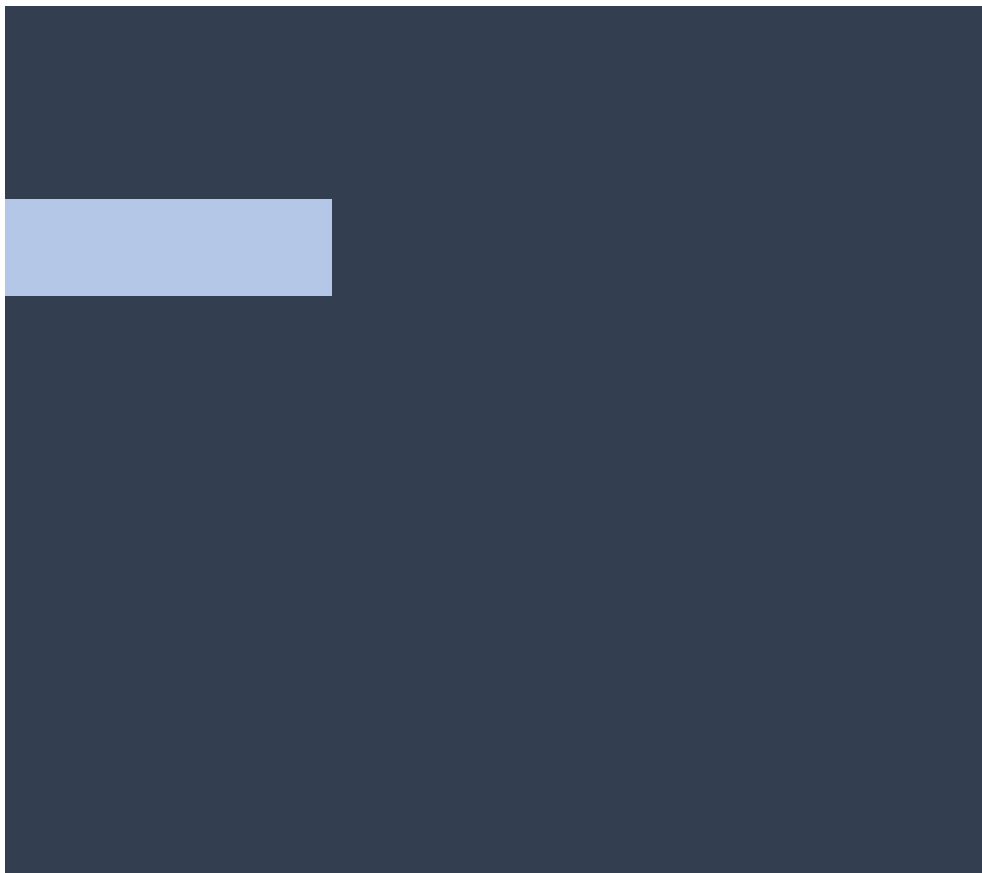




Blocking



浙江大学
ZHEJIANG UNIVERSITY

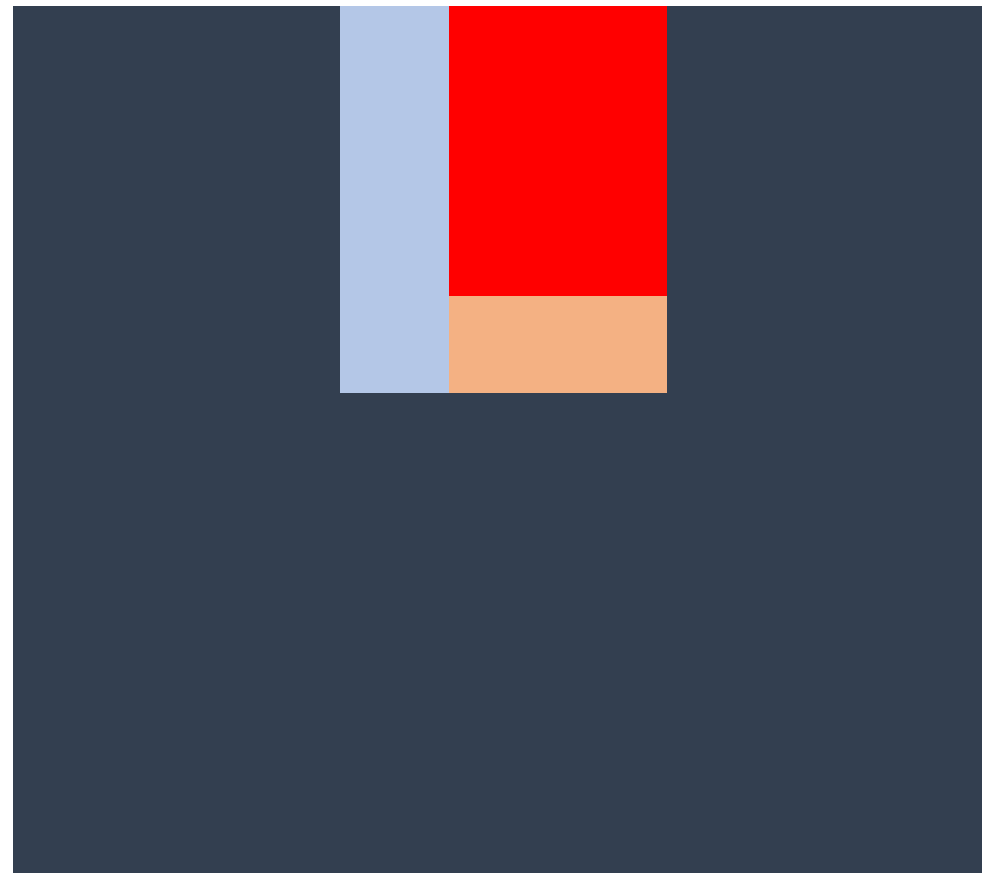
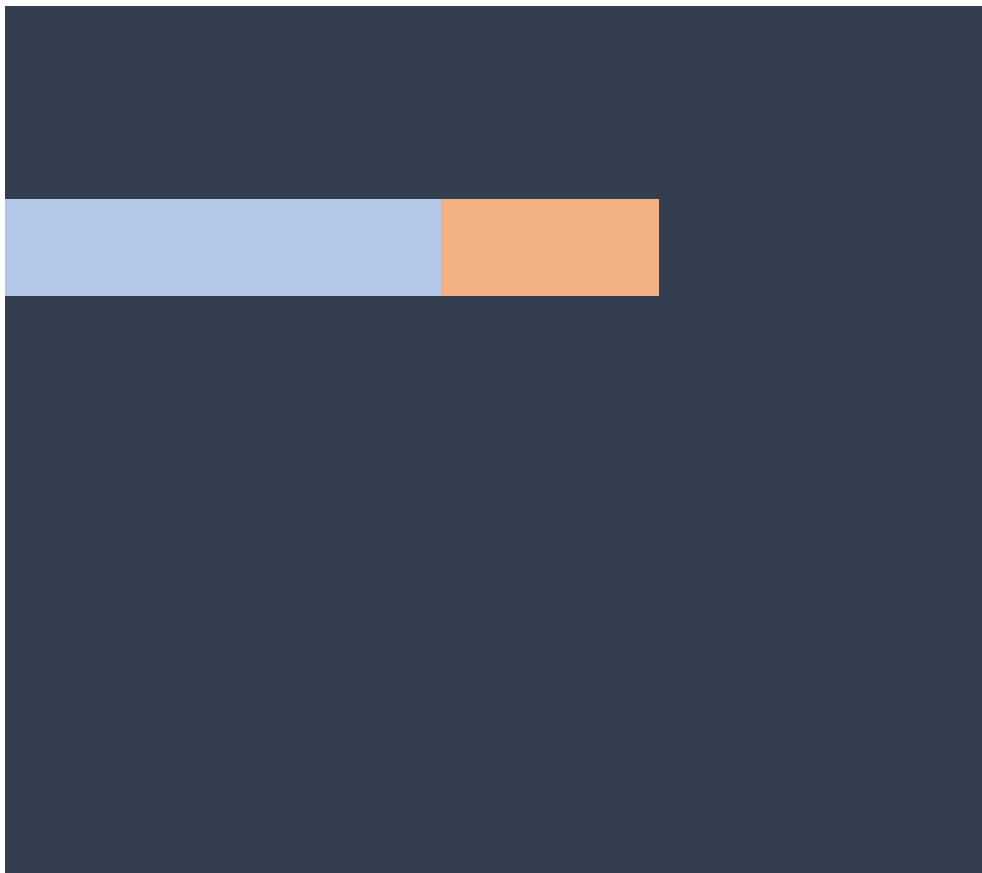




Blocking



浙江大学
ZHEJIANG UNIVERSITY

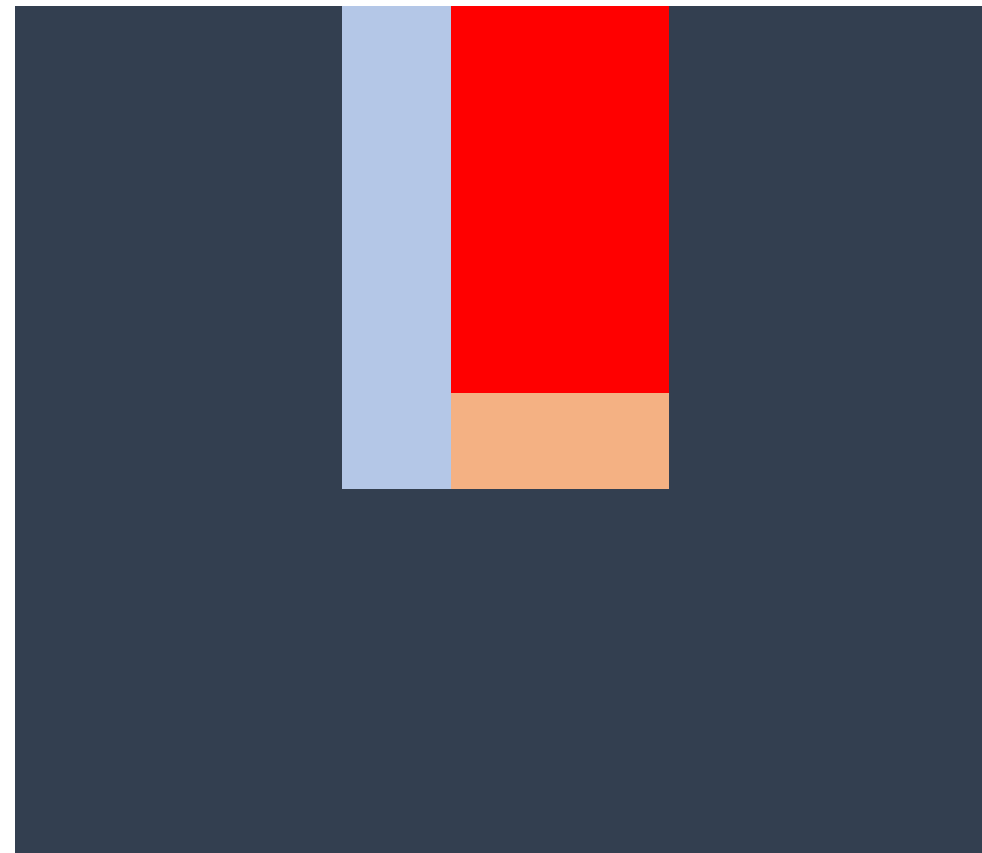
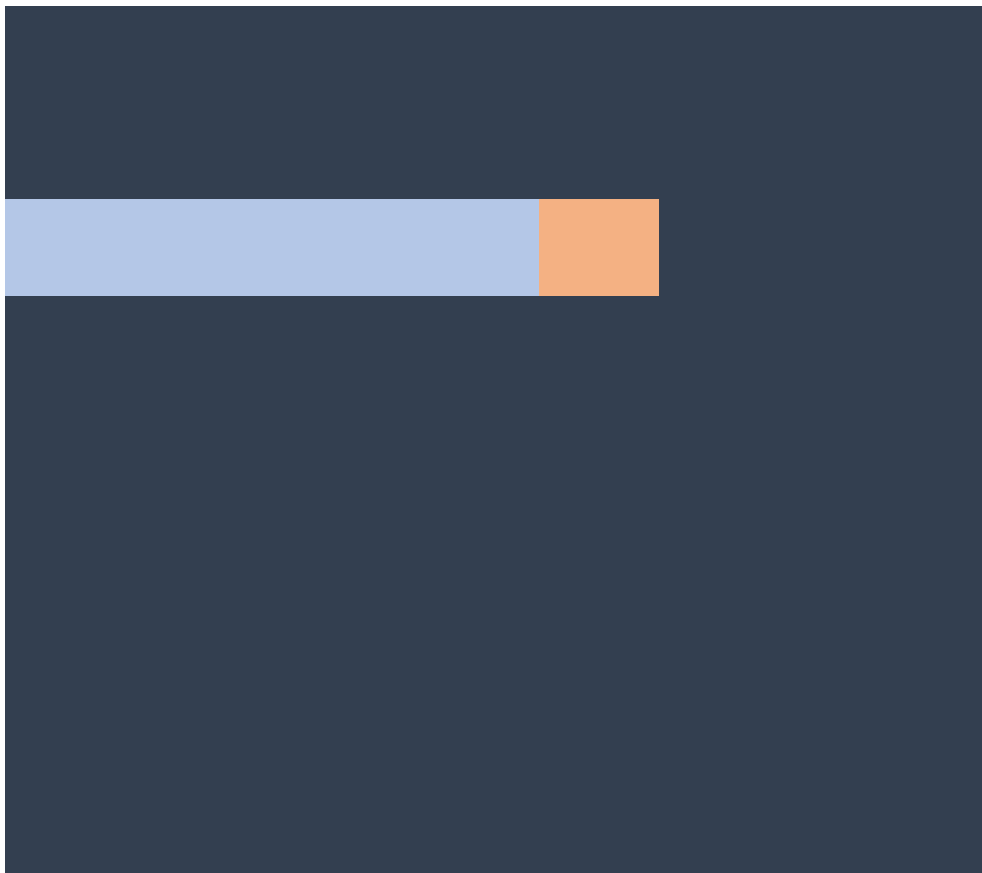




Blocking



浙江大学
ZHEJIANG UNIVERSITY

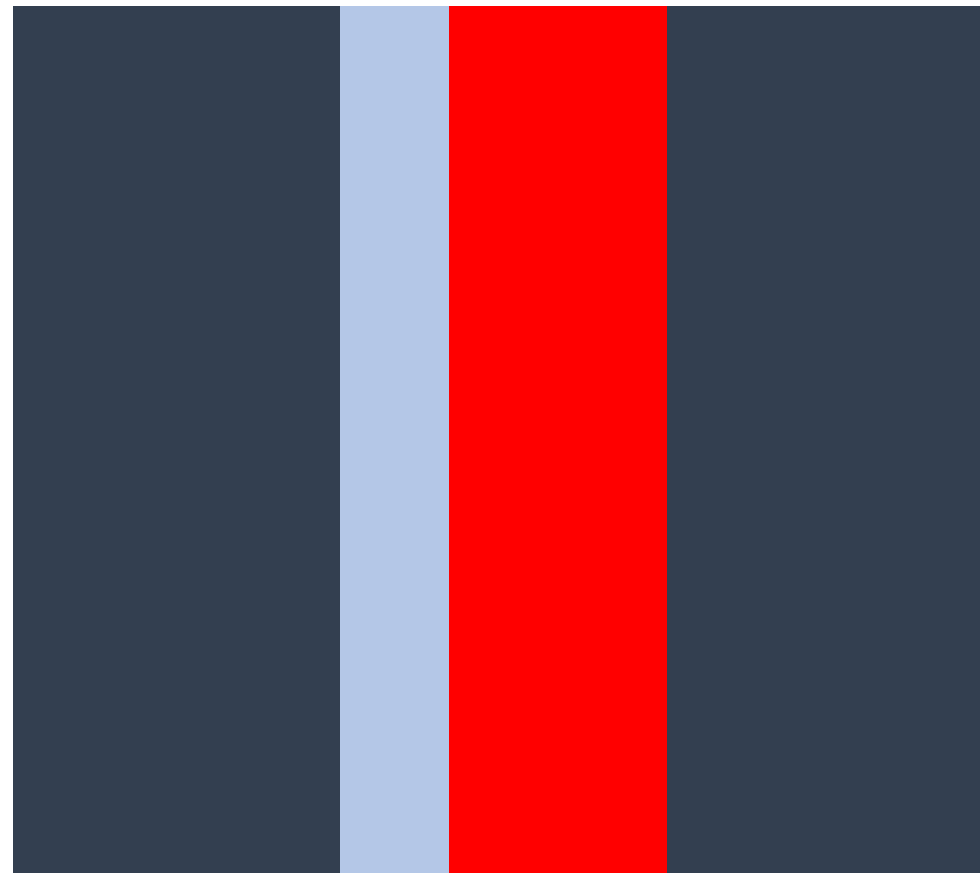




Blocking

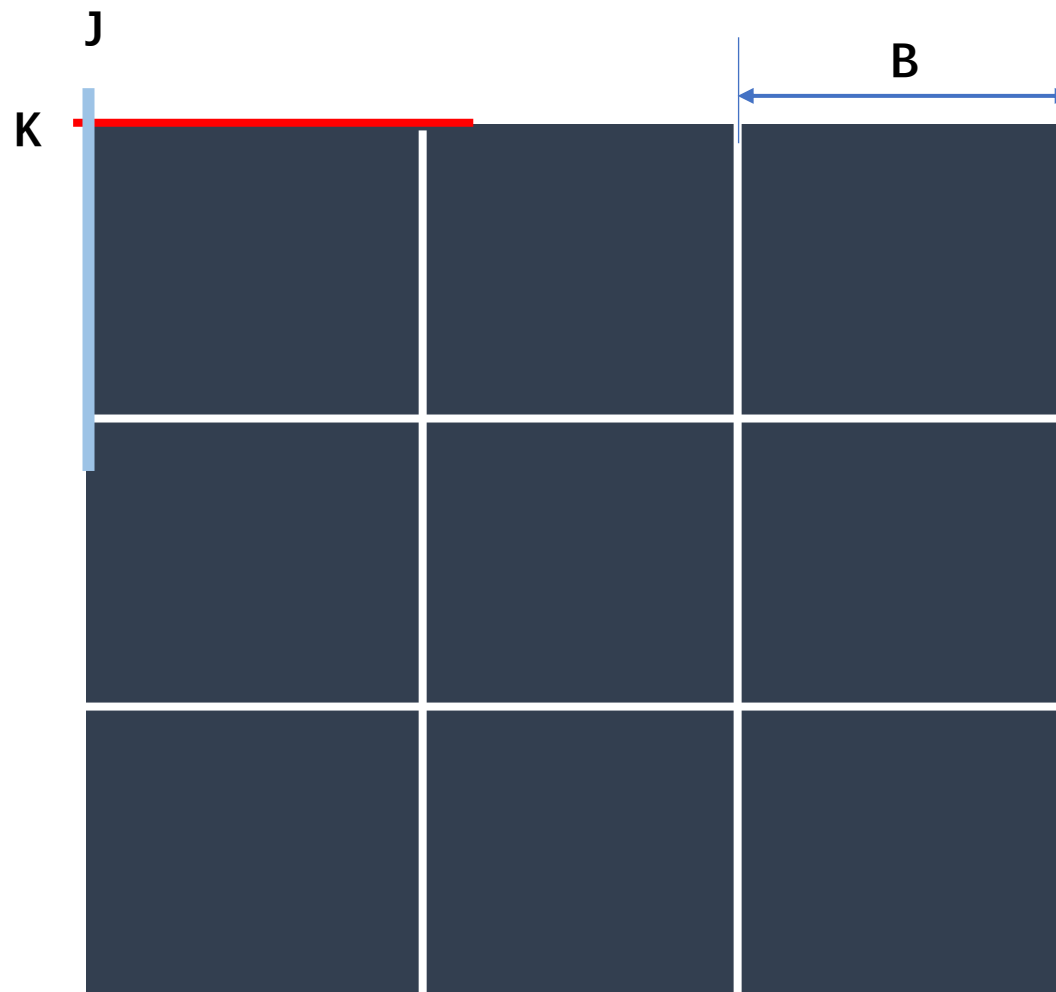
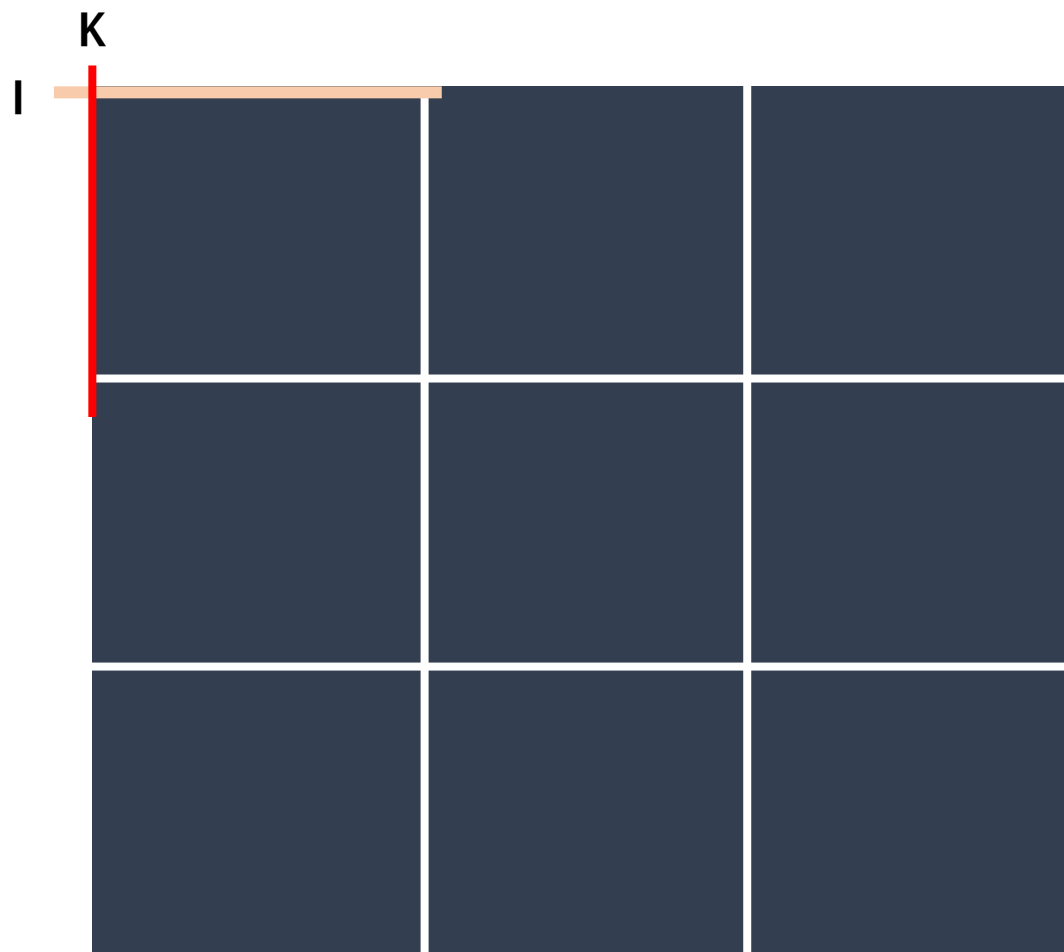


浙江大学
ZHEJIANG UNIVERSITY



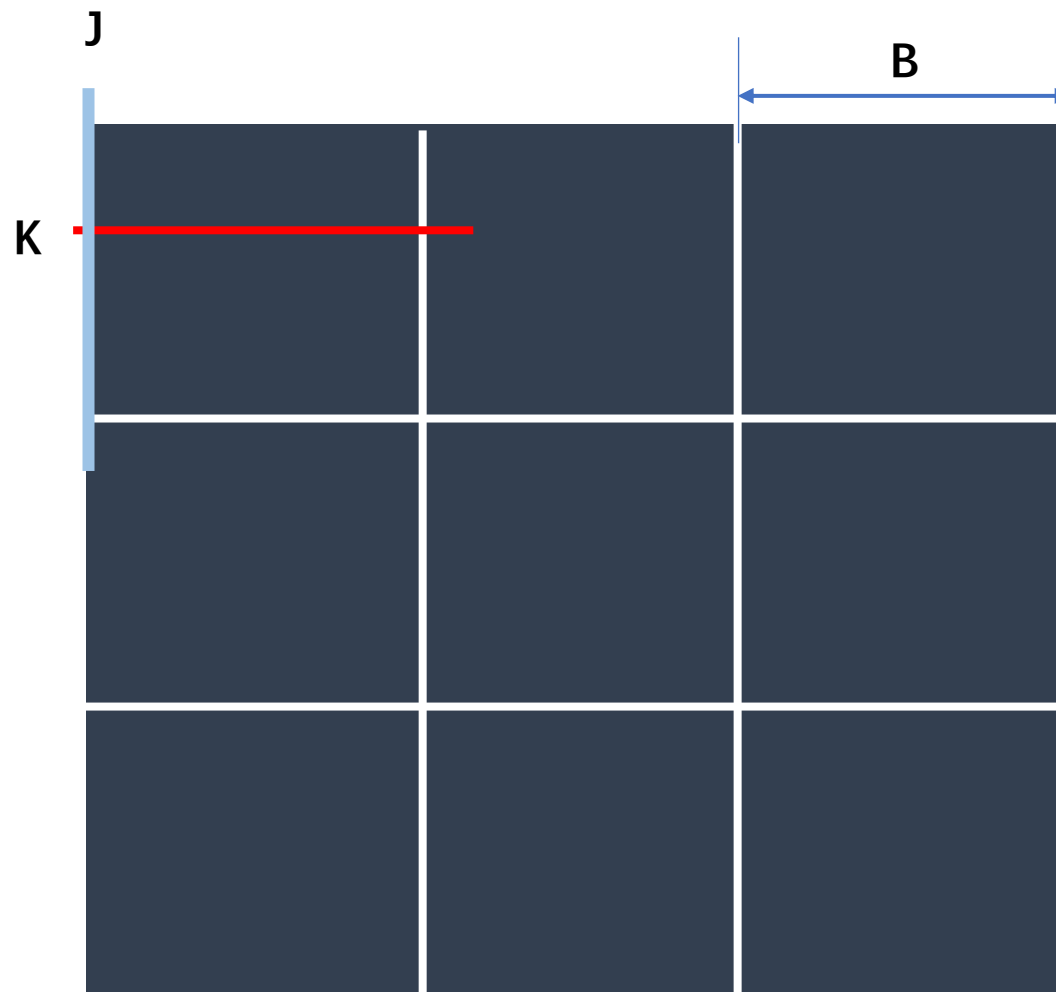
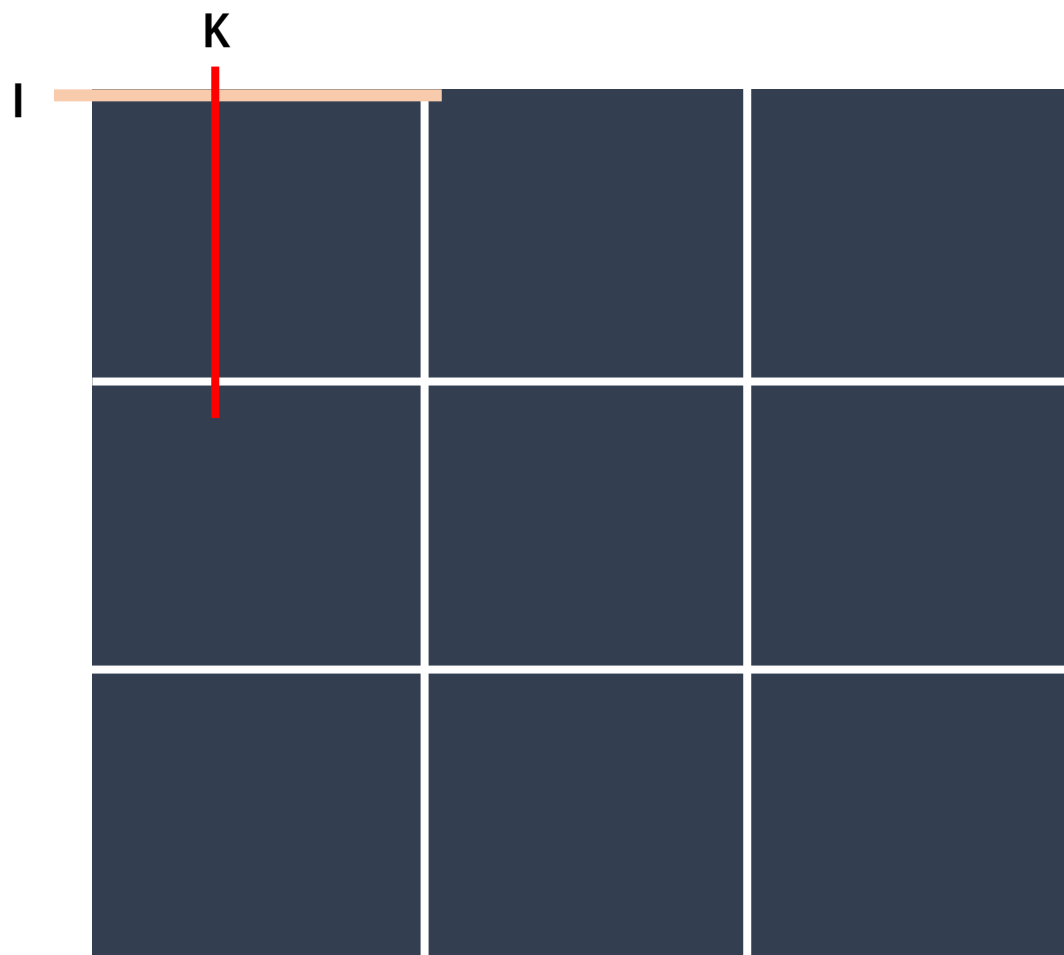


Blocking





Blocking



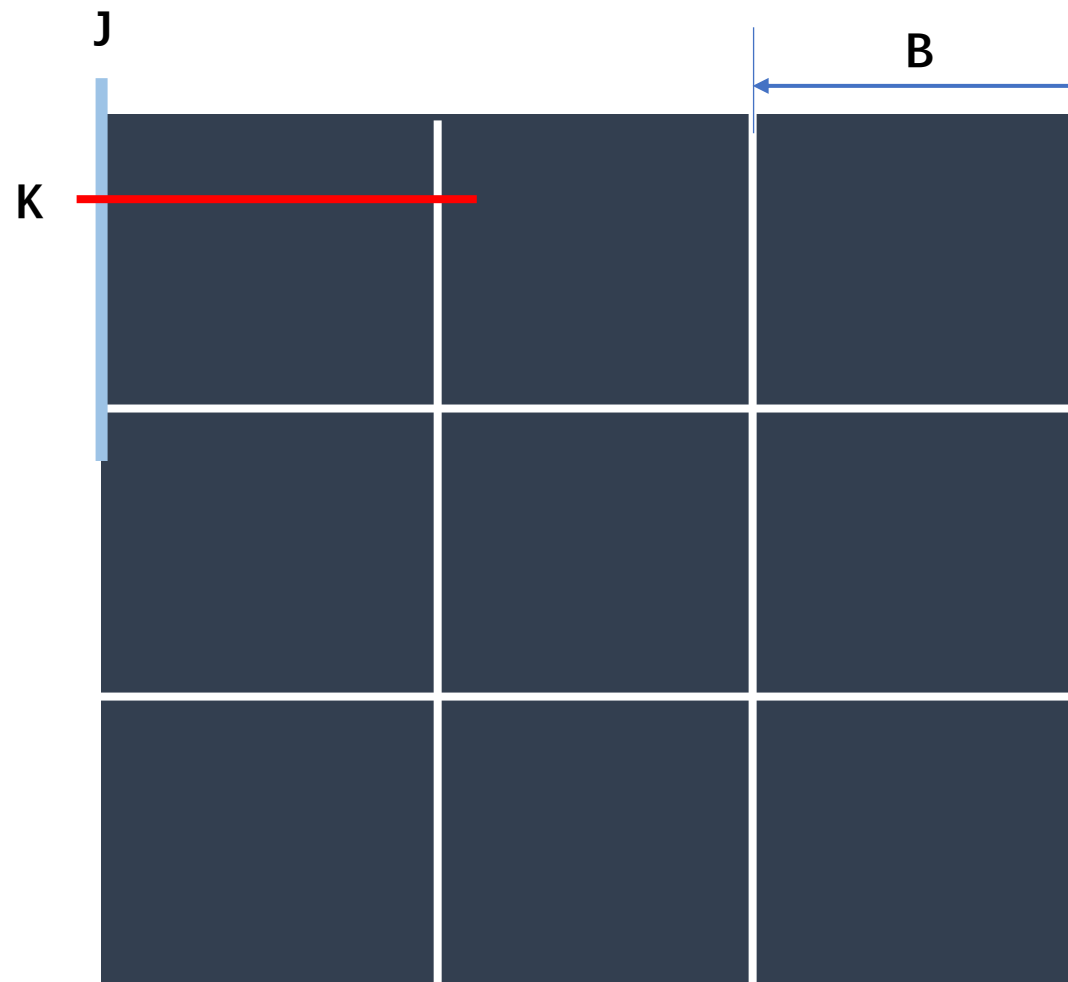
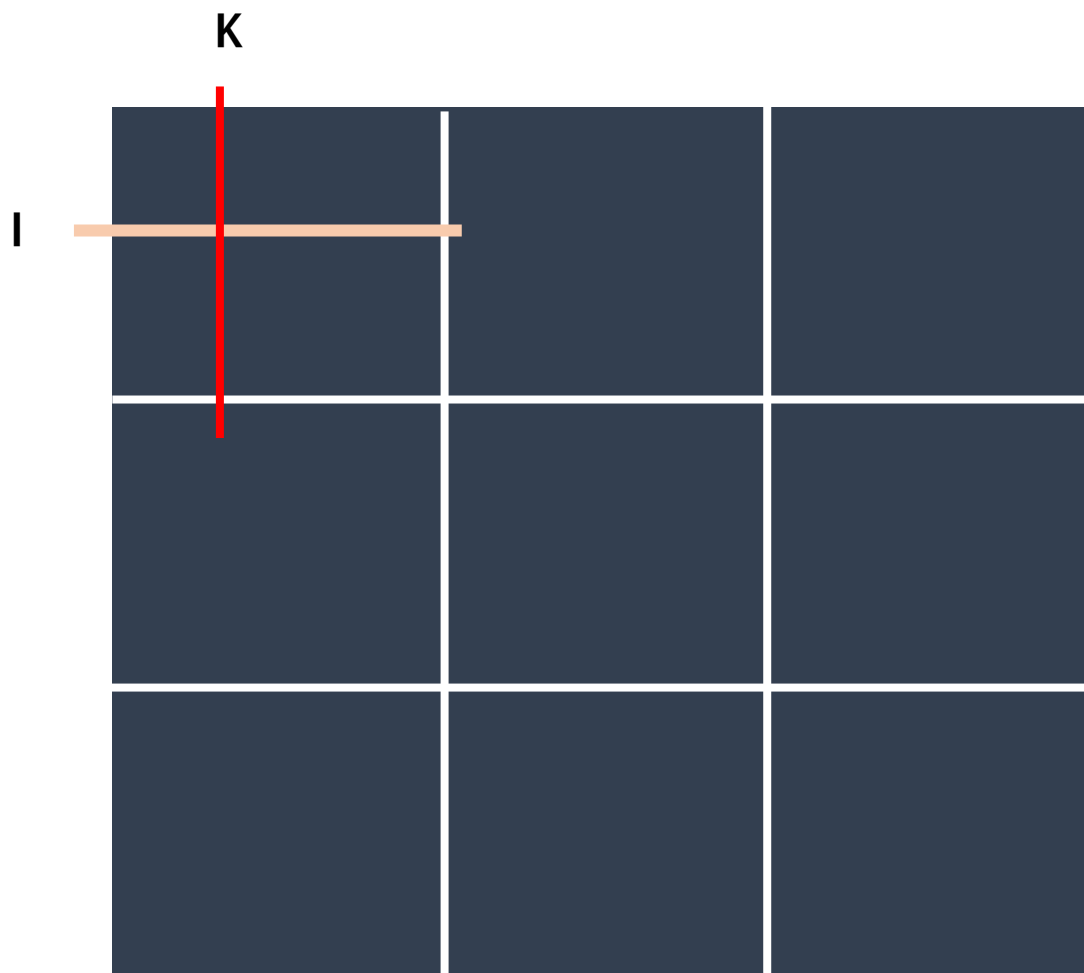


Blocking

cache

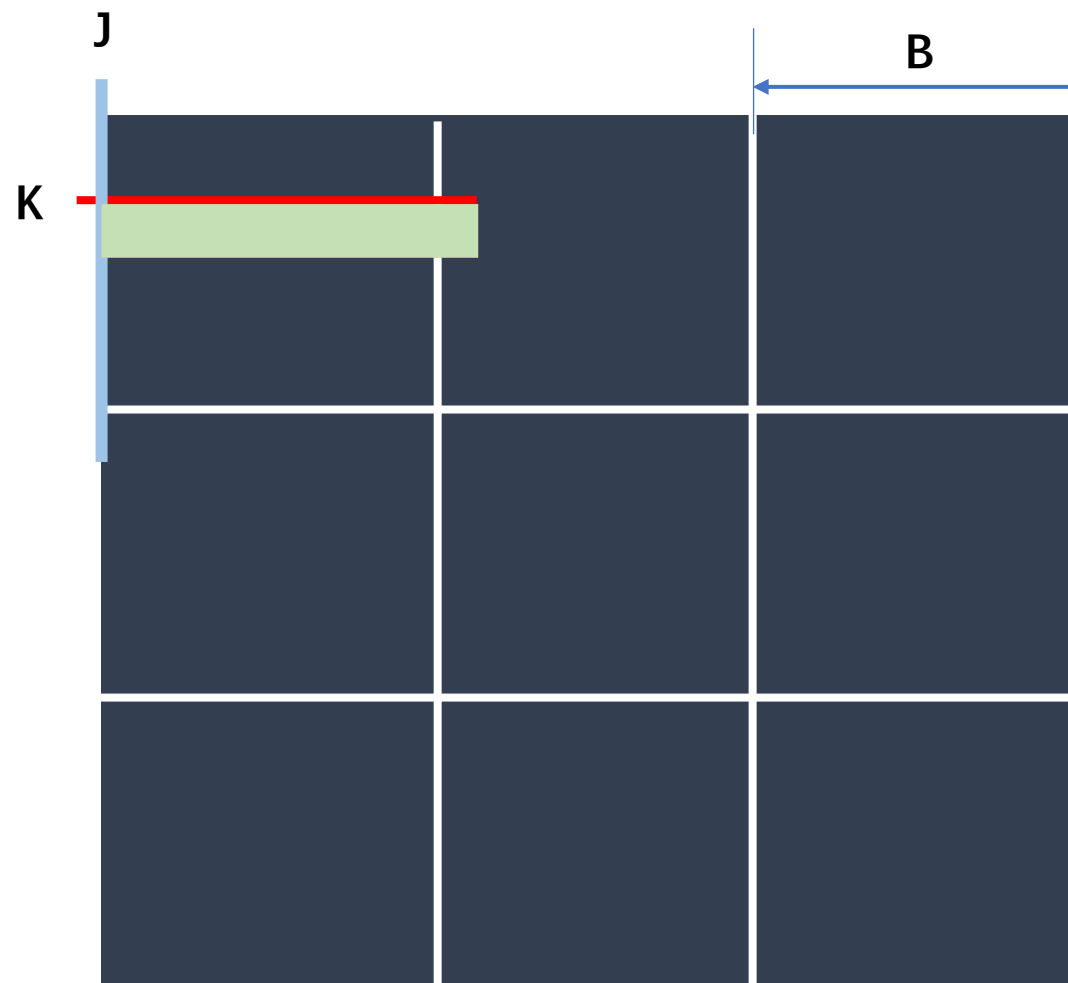
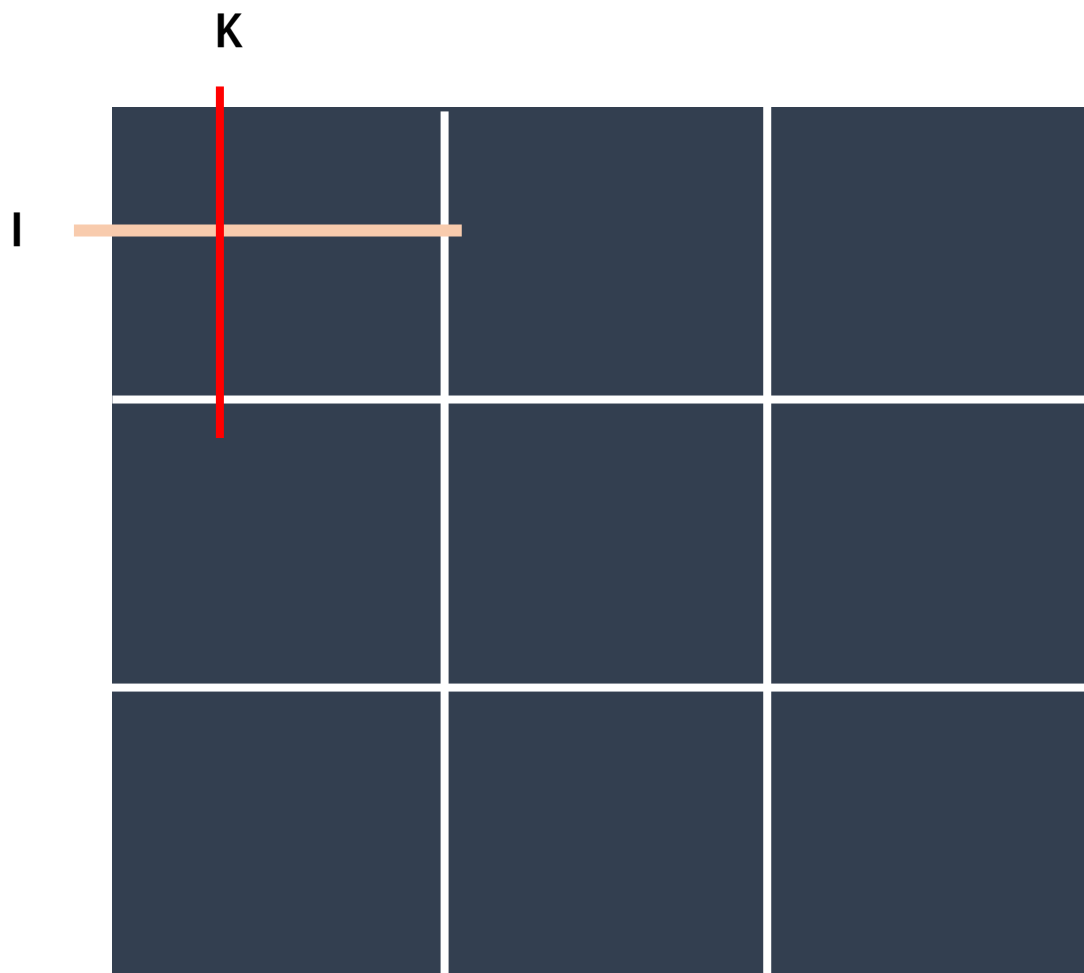


浙江大学
ZHEJIANG UNIVERSITY





Blocking





Blocking

- 在计算时间复杂度上，两者相同，都是 $O(N^3)$
- 简单估算一下Miss次数，从 $O(N^3)$ 变为 $O(N^3/B)$

```
/* Before */
for (i = 0; i < N; i = i+1)
    for (j = 0; j < N; j = j+1){
        r = 0;
        for (k = 0; k < N; k = k+1)
            r = r + y[i][k]*z[k][j]; // 关键点: z[k][j]的访问
        x[i][j] = r;
    }
```

```
/* After */
for (int kk = 0; kk < n; kk += B) {
    int k_end = min(kk + B, n);
    for (int ii = 0; ii < n; ii += B) {
        int i_end = min(ii + B, n);
        for (int jj = 0; jj < n; jj += B) {
            int j_end = min(jj + B, n);
            // 上面三个循环用于遍历分块
            // 处理分块内部计算
            for (int k = kk; k < k_end; ++k) {
                for (int i = ii; i < i_end; ++i) {
                    float yval = y[i*n + k];
                    for (int j = jj; j < j_end; ++j) {
                        x[i*n + j] += yval * z[k*n + j];
                    }
                }
            }
        }
    }
}
```



Blocking测试程序



实际上, L1 Cache大小能包含矩阵多行的数据, 所以实际结果会比理论结果小

N	B	Naive(s)	Blocked(s)	Speedup	Status
128	16	0.013	0.007	1.86x	PASS
128	32	0.013	0.007	1.81x	PASS
128	48	0.013	0.007	1.87x	PASS
128	64	0.013	0.007	1.90x	PASS
256	16	0.110	0.059	1.86x	PASS
256	32	0.111	0.060	1.84x	PASS
256	48	0.109	0.056	1.96x	PASS
256	64	0.111	0.057	1.95x	PASS
512	16	0.932	0.473	1.97x	PASS
512	32	0.915	0.482	1.90x	PASS
512	48	0.914	0.476	1.92x	PASS
512	64	0.912	0.474	1.92x	PASS
1024	16	7.578	3.905	1.94x	PASS
1024	32	7.623	3.937	1.94x	PASS
1024	48	7.633	3.925	1.94x	PASS
1024	64	7.581	3.882	1.95x	PASS

$$Sp_{overall} = \frac{T_{old}}{T_{new}} = \frac{1}{(1-f) + \frac{f}{Sp}}$$

$$Sp \approx B$$

$$f \approx 60\%$$

$$Sp_{overall} \approx 2.5$$

测试要点

- 使用windows api中的高精度计时器 QueryPerformanceFrequency() 和 QueryPerformanceCounter()
- 阻止编译器优化 `#pragma optimize("", off)`
- 按照一维数组方式构建二维数组, 保证Row Major Order的特性
- 禁用自动向量化, 禁用编译器内联优化等
- 内存64位对齐, 减少跨缓存行访问对实验造成的干扰

14. Hardware Prefetching

15. Compiler-Controlled Prefetching to Reduce Miss Penalty or Miss Rate



Hardware Prefetching

硬件自动检测数据访问模式（如顺序访问），并提前将未来可能需要的数据预取到缓存中，无需程序员干预。



Compiler-Controlled Prefetching

编译器在分析代码后，显式地插入预取指令，指示CPU提前加载特定数据，需要对代码进行静态分析。



Pros, Cons & Scenarios

Advantages: 主动将数据预取到缓存，有效降低缓存失效率和失效开销，提升程序运行速度。

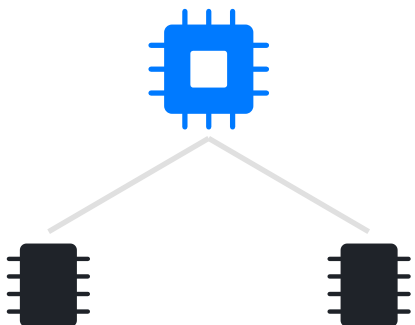
Disadvantages: 预测错误会引入不必要的内存访问，浪费总线带宽和CPU功耗，反而降低性能。

Scenarios: 适用于具有高度可预测访问模式的程序，例如数组遍历、矩阵运算等顺序或步长访问场景。

“核心思想：兵马未动，粮草先行——在CPU真正需要数据前提前加载”

16. Multiple Memory Buses and Memory Modules to Increase Bandwidth

Parallel Access Architecture



CPU accessing multiple modules via separate buses

核心原理

增加内存总线数量，CPU通过多条总线并行访问不同模块，提升总带宽。

主要优点

显著增加内存带宽，有效缓解CPU与内存之间的数据传输瓶颈。

潜在缺点

硬件成本上升，同时增加了主板布线和设计的复杂度。

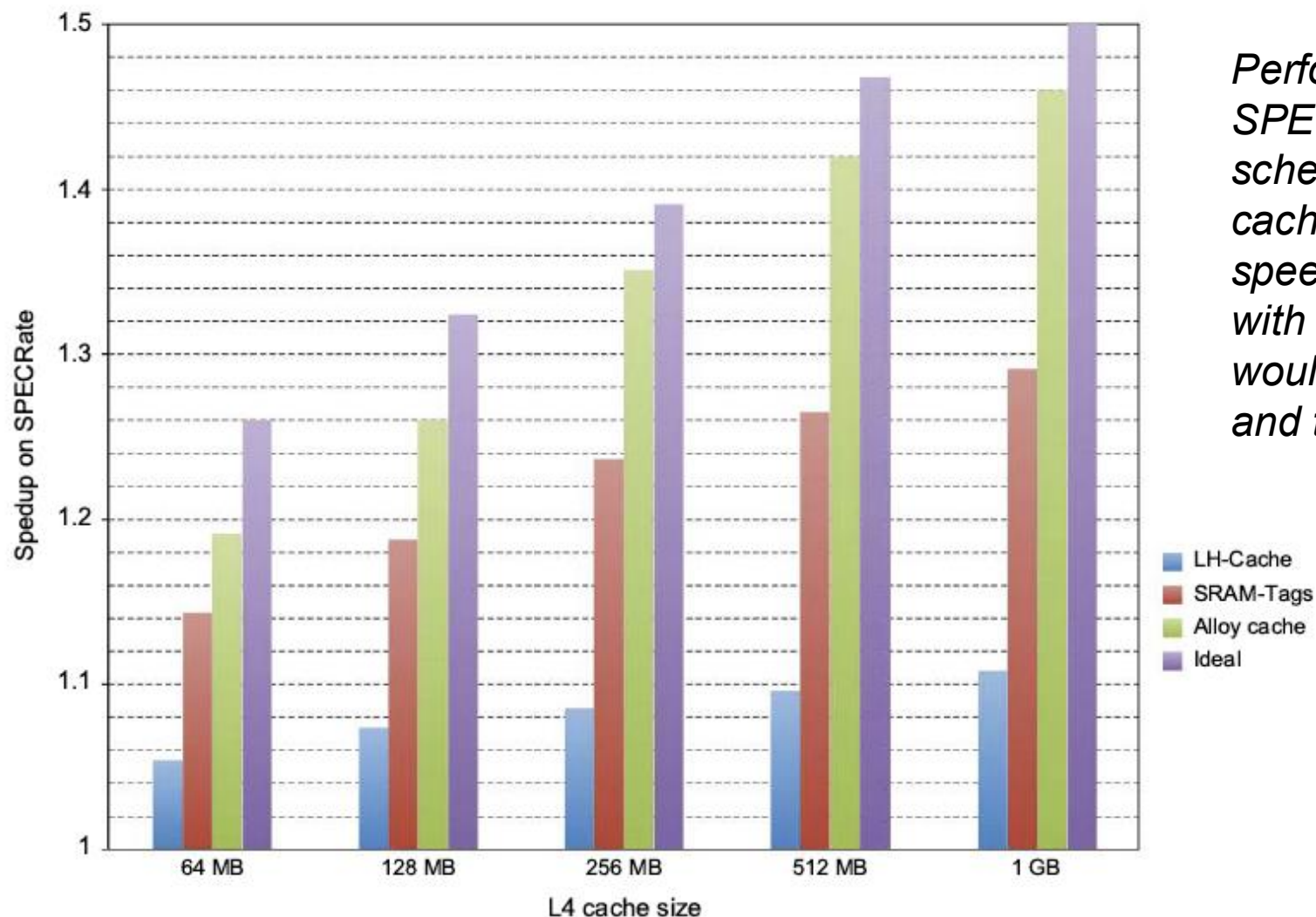
适用场景

适用于对内存带宽有极高需求的服务器、工作站及高性能计算(HPC)系统。



关键总结：内存总线如同数据高速公路，增加车道（多总线）是解决高并发数据访问拥堵的有效手段。

16. Multiple Memory Buses and Memory Modules to Increase Bandwidth



Performance speedup running the SPECrate benchmark for the Loh-Hill scheme, an SRAM tag scheme, an alloy cache, and an ideal LLC (Ideal); a speedup of 1 indicates no improvement with the L4 LLC, and a speedup of 2 would be achievable if L4 were perfect and took no access time.

How to Improve

Hence, there are more than 20 cache optimizations into these categories:

$$AMAT = HitTime + MissRate \times MissPenalty$$

1. Reduce the miss penalty
 - multilevel caches, critical word first, read miss before write miss, merging write buffers, and victim caches
2. Reduce the miss rate
 - larger block size, large cache size, higher associativity, way prediction and pseudo-associativity, and compiler optimizations
3. Reduce the time to hit in the cache
 - small and simple caches, avoiding address translation, pipelined cache access, and trace caches³.
4. Reduce the miss penalty and miss rate via parallelism
 - non-blocking caches, hardware prefetching, and compiler prefetching



$$AMAT = HitTime + MissRate \times \underline{MissPenalty}$$

1. Reduce the miss penalty

- multilevel caches
- critical word first
- read miss before write miss
- merging write buffers
- victim caches



$$AMAT = HitTime + \underline{MissRate} \times MissPenalty$$

2. Reduce the miss rate

- larger block size
- large cache size
- higher associativity
- way prediction and pseudo-associativity
- compiler optimizations



$$AMAT = \underline{HitTime} + MissRate \times MissPenalty$$

3. Reduce the time to hit in the cache

- small and simple caches
- avoiding address translation
- pipelined cache access
- trace caches



$$AMAT = HitTime + \underline{MissRate \times MissPenalty}$$

4. Reduce the miss penalty and miss rate via parallelism

- non-blocking caches
- hardware prefetching
- compiler prefetching



How to Improve



核心目标：降低 AMAT

缓存性能优化的核心在于降低平均访存时间，这是衡量系统效率的关键指标。



多维优化策略

涵盖降低失效率、减少失效开销、提升命中速度及增加带宽等多种维度的技术手段。



权衡与场景 (Trade-off)

每种优化技术都有其利弊，实际设计中需根据应用场景进行取舍与平衡。



组合拳效应

现代高性能处理器极少依赖单一技术，而是通过多种技术的有机结合来达到极致性能。



Example: Impact on Performance

- Assume :

- all instructions usually take 1.0 clock cycles (ignoring memory stalls).
- cache miss penalty is 200 clock cycles
- the average miss rate is 2%
- there is an average of 1.5 memory references per instruction
- the average number of cache misses per 1000 instructions is 30

With a cache CPI = 7

Without a cache CPI = $1.0 + 200 \times 1.5 = 301$

- What is the impact on performance when behavior of the cache is included? Calculate the impact using both misses per instruction and miss rate.

Answer: $\text{CPU}_{\text{time}} = (\text{CPU clock cycles} + \text{IC} \times \text{memory stall cycles/Inst}) \times \text{CycleTime}$
 $= \text{IC} \times (\text{CPI} + \text{memory stall cycles/Inst}) \times \text{CycleTime}$
 $= \text{IC} \times [1.0 + (30/1000 \times 200)] \times \text{clock cycle} = \text{IC} \times 7.00 \times \text{clock cycle}$

- Now calculating performance using miss rate, first compute memory stall cycles:

$$\text{CPU time} = \text{IC} \times \left(\text{CPI}_{\text{execution}} + \text{Miss rate} \times \frac{\text{Memory accesses}}{\text{Instruction}} \times \text{Miss penalty} \right) \times \text{Clock cycle time}$$

$$\text{CPU time}_{\text{with cache}} = \text{IC} \times [1.0 + (1.5 \times 2\% \times 200)] \times \text{Clock cycle time}$$

$$= \text{IC} \times 7.00 \times \text{Clock cycle time}$$



Summary of Cache Optimization Technology

Technique	Hit time	Miss penalty	Miss rate	Hardware complexity	Comment
Larger block size		–	+	0	Trivial; Pentium 4L2 uses 128 bytes
Larger cache size	–		+	1	Widely used, especially for L2 caches
Higher associativity	–		+	1	Widely used
Multilevel caches		+		2	Costly hardware; harder if L1 block size $\neq$ L2 block size; widely used
Read priority over writes		+		1	Widely used
Avoiding address translation during cache indexing	+			1	Widely used

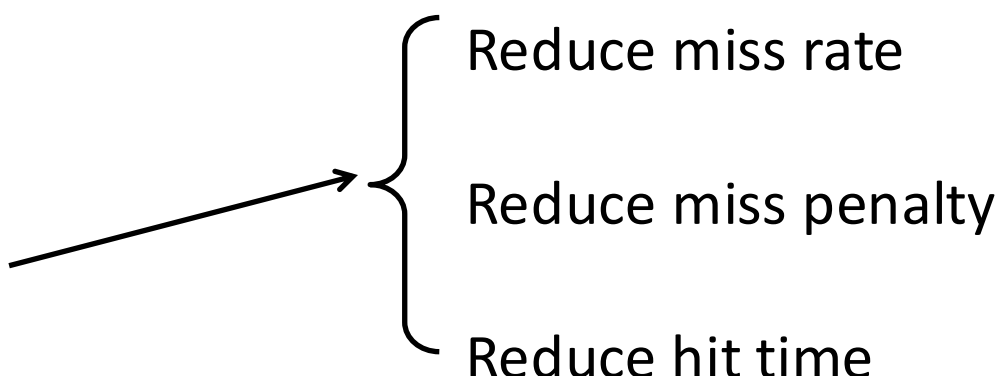


Summary of Cache Optimization Technology

Technique	Hit time	Band-width	Miss penalty	Miss rate	Power consumption	Hardware cost/complexity	Comment
Small and simple caches	+			–	+	0	Trivial; widely used
Way-predicting caches	+				+	1	Used in Pentium 4
Pipelined & banked caches	–	+				1	Widely used
Nonblocking caches		+	+			3	Widely used
Critical word first and early restart			+			2	Widely used
Merging write buffer			+			1	Widely used with write through
Compiler techniques to reduce cache misses				+		0	Software is a challenge, but many compilers handle common linear algebra calculations
Hardware prefetching of instructions and data			+	+	–	2 instr., 3 data	Most provide prefetch instructions; modern high-end processors also automatically prefetch in hardware
Compiler-controlled prefetching			+	+		3	Needs nonblocking cache; possible instruction overhead; in many CPUs
HBM as additional level of cache		+/-	–	+	+	3	Depends on new packaging technology. Effects depend heavily on hit rate improvements



Summary

- Memory hierarchy
 - From single level to multi level
 - Evaluate the performance
 - Cache basic knowledge
 - Mapping rules
 - Access method
 - Replacement algorithm
 - Write strategy
 - Cache performance analysis
 - Cache Security
- 
- Reduce miss rate
 - Reduce miss penalty
 - Reduce hit time

